



**INSTITUT SUPERIEURE
POLYTECHNIQUE DE
MADAGASCAR**



**MINISTERE DE LA
DECENTRALISATION ET DE
L'AMENAGEMENT DU
TERRITOIRE**



**OBSERVATOIRE DU
TERRITOIRE**

**MEMOIRE DE FIN D'ETUDES DU PREMIER CYCLE EN VUE DE
L'OBTENTION DU DIPLOME DE LICENCE EN INFORMATIQUE ET
TELECOMMUNICATIONS**

**PARCOURS : INFORMATIQUE, MULTIMEDIA, TECHNOLOGIE DE
L'INFORMATION, DE LA COMMUNICATION ET INTELLIGENCE
ARTIFICIELLE**

(IMTICIA)

**REFONTE ET OPTIMISATION DE LA PLATEFORME
WEB DE L'OBSERVATOIRE DU TERRITOIRE**

Présenté par :

- Monsieur ANDRIAMAHENINA Aina Loïc
- Monsieur RANDRIANANTENAINA Jean Carlos

Membres du Jury :

Président du Jury : Professeur RABOANARY Julien Amédée, Recteur de l'ISPM

Encadreur Pédagogique : Monsieur RAJAONERA Christiano Andriamialy, Enseignant-Chercheur à l'ISPM

Encadreur Professionnel : Monsieur RAJAONARISON Eli Olivier, Chef de Service de l'Information et de Diffusion des Données Territoriales.

Septembre 2025

REMERCIEMENTS

Nous tenons à profiter de cette section pour exprimer notre profonde reconnaissance envers toutes les personnes qui ont contribué, directement ou indirectement, à la réalisation de ce mémoire.

Pour cela nous voulons exprimer nos sincères remerciements :

- ❖ A son Excellence Monsieur ANDRIATSITOHAINA Naina, Ministre du Ministère de la Décentralisation et de l'Aménagement du Territoire pour l'opportunité qui nous a été offerte d'effectuer notre stage au sein de son département ;
- ❖ Au Professeur Julien Amédée RABOANARY, Recteur de l'Institut Supérieur Polytechnique de Madagascar ;
- ❖ A Monsieur RAJAONARISON Eli Olivier, Chef de Service de l'Information et de Diffusion des Données Territoriales et notre encadreur professionnel pour ses précieux conseils, sa disponibilité, et son encadrement tout au long de ce projet. Son expertise et son soutien nous ont été d'une aide inestimable ;
- ❖ A Monsieur RAJAONERA Christiano Andriamialy, enseignant-chercheur à l'ISPM et notre encadreur pédagogique pour son aide, ses conseils avisés, et sa disponibilité tout au long de ce travail. Son soutien a été essentiel à la réussite de ce mémoire ;
- ❖ A Monsieur RAFINDRALAMBO Guy Emmanuel, directeur de l'Observatoire du Territoire, direction rattachée au Ministère de la Décentralisation et Aménagement du Territoire ;

Nos humbles remerciements également :

- ❖ A tous les Professeurs de l'Institut Supérieur Polytechnique de Madagascar de leurs enseignements et leur soutien académique durant nos 3 années de parcours ;
- ❖ A nos familles respectives qui nous ont soutenu moralement et financièrement

Enfin, nous remercions toutes les personnes qui ont, de près ou de loin, contribué à la réalisation de ce mémoire.

AVANT-PROPOS

Depuis l'année universitaire 2008-2009, l'Institut Supérieur Polytechnique de Madagascar (ISPM) a adopté le système LMD (Licence Master Doctorat) visant à accentuer les recherches des étudiants et ainsi suivre une norme instaurée à l'étranger et qui a déjà fait ses preuves. Pour y parvenir, chaque étudiant doit choisir un thème, l'approfondir au bout. Un mémoire sera rédigé et soutenu en vue de l'obtention du Diplôme de Licence.

ANDRIAMAHENINA Aina Loïc et RANDRIANANTENAINA Jean Carlos du parcours : Informatique, Multimédia, Technologie de l'information et de la Communication et Intelligence Artificielle, avons choisi l'Observatoire du Territoire pour notre stage car on veut mettre nos connaissances en informatique au service de l'Etat. Comme thème développé tout au long de ce mémoire, nous avons choisi "Refonte et Optimisation de la Plateforme Web de l'Observatoire du territoire".

LISTE DES ABRÉVIATIONS

AEE : Agriculture Et Elevage

API : Application Programming Interface

CAA : Commerce et Administration des Affaires

CRUD: Create Read Update Delete

CSS : Cascading Style Sheets

DTJA : Droit Technique Juridique des Affaires

EMP : Economie et Management de Projet

EMII : Electro-Mécanique et Informatique Industrielle

ESIIA : Electronique Systèmes Informatiques et Intelligence Artificielle

FIC : Finances et Comptabilités

GCA : Génie Civil et Architecture

HTML : HyperText Markup Language

HTTP: HyperText Transfer Protocol

IAA : Industrie Agroalimentaire

ICMP : Industries Chimiques, Minières et Pétrolières

IGGLIA : Informatique de Gestion, Génie Logiciel et Intelligence Artificielle

IMTICIA : Informatique Multimédia, Technologie de l'Information et de la Communication et Intelligence Artificielle

ISAIA : Informatique Statistique Appliqué et Intelligence Artificielle

ISPM : Institut Supérieur Polytechnique de Madagascar

JS : Javascript

MDAT: Ministère de la Décentralisation et de l' Aménagement du Territoire

OT: Observatoire du Territoire

PIP : Pharmacologie et Industries Pharmaceutiques

PUDE : Plan d'Urbanisme de Détail

PUDI : Plan d'Urbanisme Directeur

SAC : Schéma d'Aménagement Communaux

SAIC : Schéma d'Aménagement Inter-Communaux

SASS : Syntactically Awesome StyleSheets

SGBD : Système de Gestion de Base de Données

SGBDR : Système de Gestion de Base de Données Relationnelle

SNAT : Schéma National de l'Aménagement du Territoire

SQL : Structured Query Language

SRAT : Schéma Régional de l'Aménagement du Territoire

TEE : Tourisme et Environnement

TEH : Tourisme et Hôtellerie

UI : User Interface / Interface Utilisateur

UML : Unified Modeling Language

UUID : Universally Unique Identifier

UX : User Experience / Expérience Utilisateur

LISTE DES FIGURES

Figure 1 : Logo ISPM.....	7
Figure 2 : Coursus de l'ISPM.....	8
Figure 3 : Organigramme de k'ISPM	11
Figure 4 : Organigramme du MDAT.....	13
Figure 5 : Accueil du site de l'OT (partie 1).....	15
Figure 6 : Accueil du site de l'OT (partie 2).....	16
Figure 7 : Extrait de page du site de l'OT.....	17
Figure 8 : Page de présentation.....	18
Figure 9 : Navigation du site	19
Figure 10 : Diagramme de cas d'utilisation (partie 1)	21
Figure 11 : Diagramme de cas d'utilisation (partie 2)	22
Figure 12 : Diagramme de classe	23
Figure 13 : Diagramme de séquence.....	24
Figure 14 : Maquette de l'accueil.....	26
Figure 15 : Maquette de la page des outils.....	27
Figure 16 : Maquette de la version mobile de l'accueil.....	28
Figure 17 : Logo HTML	31
Figure 18 : Logo CSS	31
Figure 19 : Logo Bootstrap.....	32
Figure 20 : Logo JavaScript	33
Figure 21 : Logo NodeJS.....	34
Figure 22 : Logo VueJS	35
Figure 23 : Logo ExpressJS.....	36
Figure 24 : Logo SQL.....	36
Figure 25 : Logo PostgreSQL.....	37
Figure 26 : Logo VS Code.....	38
Figure 27 : Logo Adobe XD.....	36
Figure 28 : Logo Vetur	39
Figure 29 : Liste des Tables	42
Figure 30 : Champs de la table plan_urbanisme.....	43
Figure 31 : Champs de la table schema_reg	44

Figure 32 : Champs de la table procédures.....	45
Figure 33 : Champs de la table publications.....	46
Figure 34 : Champs de la table statistiques	47
Figure 35 : Champs de la table these_mem	48
Figure 36 : Champs de la table texte_lois.....	49
Figure 37 : Champs de la table admins	50
Figure 38 : Architecture du backend	51
Figure 39 : Imports.....	52
Figure 40 : Fonction getAll	52
Figure 51 : Fonction getByColumnValue	53
Figure 42 : Fonction addOne.....	53
Figure 43 : Fonction deleteById.....	53
Figure 44 : Fonction updateById	54
Figure 45 : Fonction findAll	55
Figure 46 : Fonction findByColumnValue	55
Figure 47 : Fonction remove.....	56
Figure 48 : Fonction addPlan	56
Figure 49 : Fonction modifyPlan	57
Figure 50 : Routes de la table plan_urbanisme	57
Figure 51 : Configuration de la base de données.....	59
Figure 52 : Extrait de code VueJS – Partie HTML	59
Figure 53 : Extrait de code VueJS de l’Accueil.....	60
Figure 54 : Import des pages dans routes.js.....	61
Figure 55 : Définition des routes des pages	61
Figure 56 : Template de la page PUDé(extrait)	62
Figure 57 : CSS de la page PUDé (extrait).....	63
Figure 58 : Script de la page PUDé (extrait).....	63
Figure 59: Partie HTML du formulaire de login.....	64
Figure 60: Partie JavaScript du formulaire de login	65
Figure 61: HTML du tableau d’affichage (Partie 1)	66
Figure 62: HTML du tableau d’affichage (Partie 2)	66
Figure 63: Code de la fonction de recherche.....	67
Figure 64: Code d’affichage, de modification et de suppression	68
Figure 65: Partie HTML du formulaire d’ajout.....	69
Figure 66: Javascript du formulaire d’ajout (Partie 1)	70

Figure 67: Javascript du formulaire d'ajout (Partie 2)	70
Figure 68: Partie HTML du formulaire de modification	71
Figure 69: Javascript du formulaire de modification (Partie 1)	72
Figure 70: Javascript du formulaire de modification (Partie 2)	72
Figure 71 : Interface de la page Guide	73
Figure 72 : Interface de la page Outils de Planification	74
Figure 73 : Interface de la page Statistiques	75
Figure 74 : Contenu de chaque bouton déroulant.....	75
Figure 75 : Connexion à l'admin.....	76
Figure 76 : Données dans la table plan_urbanisme	77
Figure 77 : Champs de recherche	78
Figure 78 : Formulaire de modification	79
Figure 79 : Confirmation de suppression.....	80
Figure 80 : Ajout de données	81

LISTE DES TABLEAUX

Tableau n° 1: Habilitation de toutes les formations à l'ISPM par le MESupReS	5
Tableau n° 2: Performances et prix obtenus par ISPM	10

SOMMAIRE

INTRODUCTION.....	1
PARTIE I : PRESENTATION GENERALE	
Chapitre I : A propos de l'ISPM	2
Chapitre II : A propos de la Direction au sein du MDAT	12
Chapitre III : Présentation et Choix du Projet	15
PARTIE II : CONCEPTION ET DÉVELOPPEMENT	
Chapitre IV : Conception du Projet	21
Chapitre V : Développement du Projet.....	31
Chapitre VI : Interface et Fonctionnement du Logiciel.....	74
PARTIE III : RÉSULTATS ET DISCUSSIONS	
Chapitre VII : Approche Qualitative du Projet	82
Chapitre VIII : Futures Améliorations du Logiciel.....	85
CONCLUSION	87

INTRODUCTION

Dans un monde de plus en plus connecté, les plateformes web jouent un rôle central dans les activités quotidiennes. Les utilisateurs attendent des performances optimales, navigation fluide, et une expérience personnalisée, rendant indispensable l'adaptation et l'amélioration continue de ces plateformes.

La plateforme web étudiée dans ce mémoire, bien qu'opérationnelle, présente plusieurs limites : un design obsolète, temps de chargement élevé, et une compatibilité mobile limitée. Ces problèmes nuisent à l'expérience utilisateur.

L'objectif principal de ce projet est de refondre et d'optimiser cette plateforme web afin de répondre aux attentes des utilisateurs et aux standards actuels. Plus spécifiquement, il s'agit de moderniser l'interface utilisateur, d'améliorer les performances techniques et de garantir une navigation fluide.

Pour atteindre ces objectifs, une méthodologie structurée a été adoptée. Elle inclut une analyse approfondie de l'état actuelle de la plateforme et l'utilisation d'outils modernes pour sa refonte. La validation est effectuée par des tests de performance et de satisfaction des utilisateurs.

Ce mémoire est structuré en trois parties. La première partie présente le cadre d'étude, les objectifs et le contexte du projet. La deuxième partie détaille les étapes de la refonte et de l'optimisation. Enfin, la troisième partie analyse les résultats obtenus et propose des perspectives d'améliorations futures.

PARTIE I : PRESENTATION GENERALE

Chapitre I : A propos de l'ISPM

L'Institut Supérieur Polytechnique de Madagascar (ISPM) est un institut d'enseignement, de formation et de recherche agréé et homologué par l'Etat suivant l'Arrêté N°3725 du 19 Août 1994 du ministère de l'enseignement Supérieur.

I.1. Historique :

Le Professeur RABOANARY Julien Amédée a fondé l'Institut Supérieur Polytechnique de Madagascar ou ISPM le 23 Janvier 1993. Il demeure le recteur jusqu'à ce jour.

A ses débuts, l'institut, sis à Ankadindramamy connu sous le nom d'Ecole Supérieure de Sciences et Techniques de l'Information à Madagascar ou ESSTIM ne proposait que deux filières :

- **filière Informatique de Gestion, Génie logiciel et Intelligence Artificielle (IGGLIA) :**

Toutes les entreprises ne peuvent plus se passer de l'outil informatique, surtout l'informatique appliquée à la gestion. Le parcours IGGLIA est un parcours dont l'objectif est la maîtrise de toutes les techniques informatiques relatives à la gestion des entreprises ;

- **filière Electronique, Systèmes Informatiques et Intelligence Artificielle (ESIIA) :**

L'informatique ne se résume pas seulement sur le software. L'électronique est avant tout l'origine de l'informatique. Elle est aussi à l'origine de la technologie des télécommunications. Le parcours ESIIA est un parcours qui offre l'opportunité de maîtriser l'électronique, la structure des ordinateurs et les applications fondamentales de l'informatique ;

Il a été homologué par l'Etat suivant l'Arrêté Ministériel n° 3725 du 19 Août 1994.

De là, l'institut s'est très vite développé et d'année en année de nouvelles filières ont vu le jour.

- **filière Commerce et Administration des Affaires (CAA) en 1993 :**

Madagascar, comme presque tous les pays du monde, a opté pour l'économie libérale où la concurrence, surtout sur le plan économique et commercial. A cela s'ajoute le contexte de mondialisation et de globalisation actuelle. C'est dans ce sens que l'ISPM a créé le parcours Commerce et Administration des Affaires qui est un parcours de formation fondée sur le marketing et les différentes techniques d'organisation et de management des entreprises ;

• filière Industries Agro-Alimentaire (IAA) et filière Pharmacologie et Industries Pharmaceutiques (PIP) en 1994 :

Les différentes industries agroalimentaires ont une place et un rôle important dans notre économie. L'ISPM a un objectif de former des cadres de qualité pour ces entreprises en plus L'existence des plantes médicinales endémiques est une réalité à Madagascar. L'ISPM veut mettre en valeur les propriétés thérapeutiques de ces plantes d'une manière scientifique ;

• Filière Tourisme et Environnement (TEE) en 1997 :

Madagascar est un des rares pays du monde. Il a un environnement unique et endémique par sa faune et sa flore. Le parcours Tourisme et Environnement a pour objectif d'enseigner aux étudiants la richesse de notre environnement et celle de notre civilisation ;

• Filière Electromécanique et Informatique Industrielle (EMII) en 1998 :

Toutes grandes écoles polytechniques qui se respectent doivent avoir un département d'électromécanique. D'ailleurs, la révolution industrielle n'a été possible qu'à travers la maîtrise de la mécanique et des technologies industrielles. Dans ce parcours, ISPM forme de jeunes capables de maîtriser toutes les technologies mentionnées ci-dessus auxquelles s'ajoutent l'informatique et informatique industrielle. A l'issue de leurs études, ils seront capables de participer au développement industriel de Madagascar ;

• Filière Génie Civil et Architecture (GCA) en 1999 :

La construction des infrastructures est un élément fondamental pour améliorer le niveau de vie d'un pays. C'est pourquoi ISPM forme à travers ce parcours Génie Civil et Architecture les jeunes qui vont améliorer non seulement les infrastructures, mais participer à l'aménagement urbain et rural.

• Filière Informatique Multimédia et Technologie de l'Information et de la Communication et Intelligence Artificielle (IMTICIA) en 2004 :

C'est un parcours dont la finalité est la formation des étudiants dans l'informatique multimédia, la télécommunication et d'une manière générale dans les nouvelles technologies de l'information et de la télécommunication.

• Filière Finance et Comptabilité (FIC) en 2004 aussi :

Toute entreprise, qu'elle soit publique ou privée, ne peut se passer des services offerts par le département finances et comptabilité. Ce sont les départements clés d'une entreprise. C'est dans ce sens que ce parcours a été créé. Elle a pour objectif de former des étudiants dans les différentes techniques quantitatives de la gestion des entreprises telles que les finances et la comptabilité ;

- **Filière Agriculture et Elevage (AEE) en 2004 aussi :**

Le secteur primaire est un des secteurs fondamentaux dans l'économie d'un pays. C'est dans ce sens qu'a été créé le parcours Agriculture et Elevage car il nous faut des jeunes capables d'appliquer les techniques et les technologies modernes dans le monde rural. Ils sont aussi formés au concept de l'agro-business ;

- **Filière Economie et Management de Projet (EMP) en 2009 :**

Dans ce parcours, l'ISPM veut former des économistes capables de faire des analyses économiques objectives, relationnelles et de les appliquer aux réalités des entreprises ;

- **Filière Droit et Techniques Juridiques des Affaires (DTJA) en 2009 :**

Dans ce parcours, l'ISPM veut former des juristes capables de maîtriser toutes les techniques juridiques et la fois nationales et internationales. L'originalité de ce parcours est qu'elle utilise l'outil informatique. Les techniciens du Droit formés par l'ISPM pourront travailler soit dans l'administration publique soit dans les entreprises privées ;

- **Filière Tourisme et Hôtellerie (TEH) en 2009 :**

La civilisation Malagasy est unique par l'apport des cultures asiatiques, africaines et plus tard européennes. Le parcours Tourisme et Hôtellerie a pour objectif d'enseigner la richesse de notre civilisation et la maîtrise des différentes techniques de l'art culinaire à la fois national et international ;

- **Filière Industries Chimiques, Minières et Pétrolières (ICMP) en 2009 :**

Dans le contexte actuel, tout pays qui se respecte doit avoir des jeunes capables de maîtriser les différents domaines tels que les industries Chimiques, Minières et Pétrolières. C'est dans ce sens que ce parcours a été créé ;

- **Filière Informatiques, Statistiques Appliquées et Intelligence Artificielle (ISAIA) en 2010 :**

C'est un parcours dont la finalité est la formation des étudiants dans l'informatique statistique appliquée. C'est pour cela que le déménagement dans un endroit plus vaste à Ambatomaro Antsobolo fut nécessaire.

Et depuis, aucune nouvelle filière n'a vu le jour à l'ISPM. L'Institut Supérieur Polytechnique de Madagascar (ISPM) est un institut d'enseignement, de formation et de recherche agréé et homologué par l'Etat suivant l'Arrêté N°3725 du 19 Août 1994 du ministère de l'enseignement Supérieur et habilité selon les arrêtés suivant :

Tableau n° 1: Habilitation de toutes les formations à l'ISPM par le MESupReS

DOMAINE / GRADE	MENTION	REFERENCE ET DATE
- Sciences et Technologies / Licence et Master	- Biotechnologies	Arrêté n°31172/2012-MESupRES du 05 Décembre 2012
- Sciences et Technologies / Licence - Master	- Génie Industriel - Génie Industriel	Arrêté n°1949/2013-MESupRES du 31 Janvier 2013 Arrête n°21909/2014-MESupRES du 11 Juin 2014
- Sciences et Technologies / Licence et Master	- Génie Civil et Architecture	Arrêté n°1949/2013-MESupRES du 31 Janvier 2013
- Sciences et Technologies / Licence et Master	- Informatique et Télécommunication	Arrêté n°1949/2013-MESupRES du 31 Janvier 2013
- Sciences de la société / Licence et Master	- Droit et des Technique Affaires	Arrêté n°11566/2014-MESupRES du 23 Mai 2013
- Arts, lettres et Sciences Humaines / Licence et Master	- Technique du Tourisme	Arrêté n°33213/2014-MESupRES du 04 Novembre 2014 Arrêté n°33033/2015-MESupRES du 05 Novembre 2015
- Sciences et Technologies / Licence et Master - Sciences et Technologies / Licence	- Technique de l'environnement et du Tourisme - Environnement tourisme	Arrêté n°33213/2014-MESupRES du 04 Novembre 2014 Arrêté n°33213/2014-MESupRES du 26 Décembre 2014

Source : www.ispm-edu.com

I.2. Objectif de l'ISPM :

L'ISPM a pour but de former des étudiants compétents, possédant tous les atouts et outils nécessaires dans le monde professionnel, honnêtes et possédant un esprit capable de s'adapter à n'importe quelle situation possible.

L'obtention d'un diplôme de baccalauréat (enseignement technique ou de l'enseignement général) est obligatoire pour l'entrer à l'ISPM et l'accès en première année se fait uniquement via un concours dans tous les parcours.

La pédagogie y est dirigée vers la professionnalisation basée sur une formation académique, ce qui nous distingue d'autres instituts. Avec son corps professoral expérimenté et des programmes d'études spécifiques, le niveau des étudiants de l'institut est apte à concourir avec ceux de l'extérieur.

I.3. Identification et activités :

Siège : Ambatomaro – Antsohobolo

Contact : +261 33 11 920 89

+261 34 99 782 96

+261 32 53 914 09

+261 20 26 384 08

Email : ispm@ispm-edu.com

Site web : www.ispm-edu.com

L'ISPM a une devise qui est : FAHAIZANA – FAMPANDROSOANA – FIHAVANANA (Savoir – Développement – Fraternité) et il a aussi un logo qui vante et illustre les valeurs mentionnées par sa devise :

- la toque illustre le Savoir (FAHAIZANA) donné par l'institut et reçu par les étudiants à la fin de leurs cursus,

- Madagascar dans le monde illustre le Développement (FAMPANDROSOANA) - Les mains se serrant illustrent la Fraternité (FIHAVANANA).



Figure 1: Logo ISPM

Source : Brochure de l'ISPM

I.4. Coursus à l'ISPM :

L'ISPM propose un cursus d'étude sur trois cycles qui sont :

Après l'obtention du diplôme de baccalauréat et la réussite au concours d'entrée, le Premier Cycle dure trois ans avec trois mois de stage en entreprise puis est sanctionné par le Diplôme de Licence.

Après le premier cycle, le Second Cycle dure deux ans avec six mois de stage toujours en Entreprise et est sanctionné à son terme par le Diplôme de Master 2.

Puis enfin, le Troisième Cycle est validé après au moins un an de recherche au sein de l'établissement après le second cycle et est sanctionné au final par le Diplôme d'Etudes Approfondies (DEA).

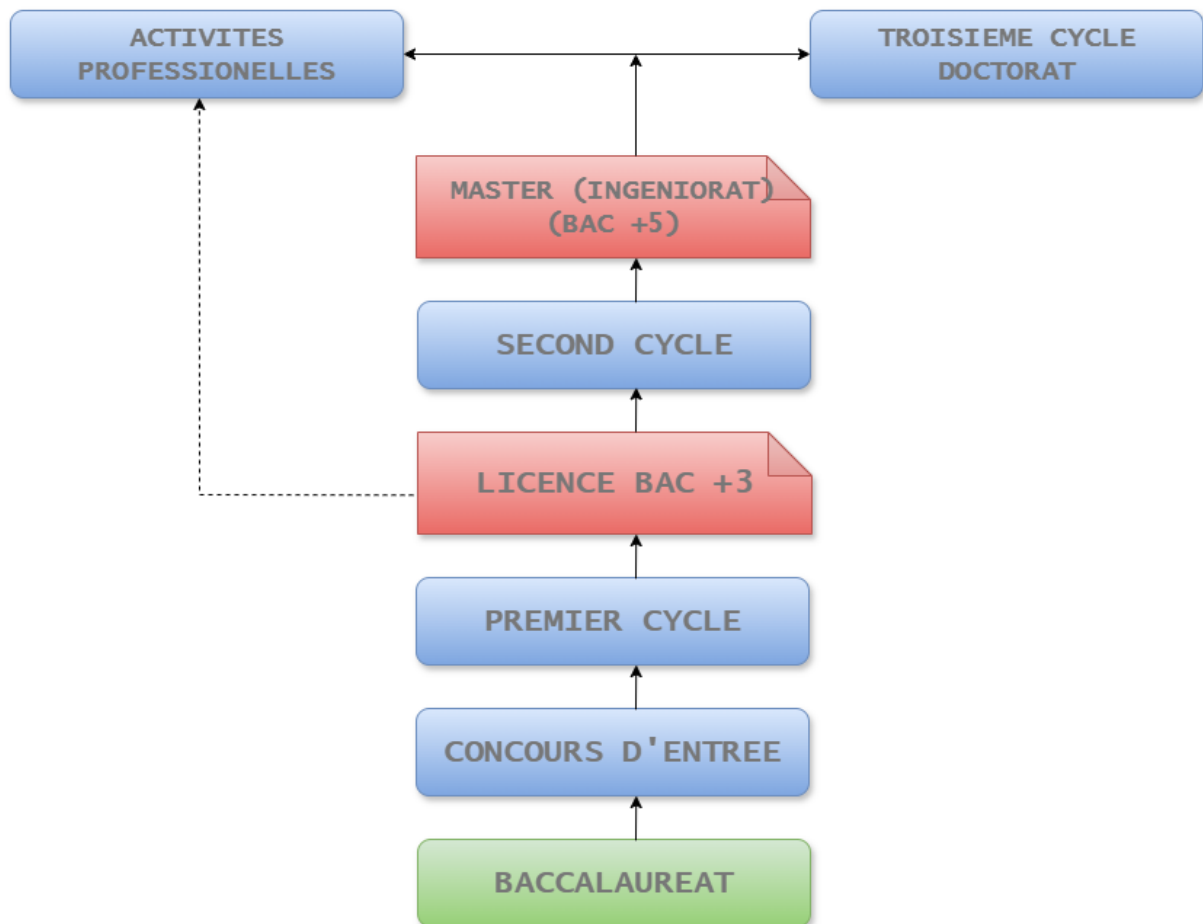


Figure 2: Cursus de l'ISPM

Source : Brochure de l'ISPM

Remarque : le choix appartient aux étudiants, à la fin des cycles, d'intégrer le monde professionnel ou de continuer les études.

Les filières précédemment citées ci-dessus sont regroupées dans cinq principaux départements qui sont :

• **Département Informatique et Télécommunications regroupant les filières:**

- Informatique de Gestion, Génie Logiciel et Intelligence Artificielle (IGGLIA) ;
- Electronique, Systèmes Informatiques et Intelligence Artificielle (ESIIA) ;
- Informatique, Multimédia, Technologie de l'Information et de la Communication, Intelligence Artificielle (IMTICIA) ;
- Informatique, Statistique Appliquée et Intelligence Artificielle (ISAIA).

• **Département Droit et Techniques des Affaires regroupant les filières :**

- Commerce et Administration des Affaires (CAA) ;
- Finances et Comptabilités (FIC) ;
- Droit et Techniques Juridiques des Affaires (DJTA) ;
- Economie et Management de Projet (EMP).

• **Département Biotechnologie et Agronomie regroupant les filières :**

- Pharmacologie et Industries Pharmaceutiques (PIP) ;
- Agriculture et Elevage (AEE) ;
- Industries Agro-Alimentaires (IAA).

• **Département Génie Industriel et Génie Civil regroupant les filières :**

- Electromécanique et Informatique Industrielle (EMII) ;
- Génie Civil et Architecture (GCA) ;
- Industries Chimiques, Minières et Pétrolières (ICMP).

• **Département Techniques du Tourisme regroupant les filières :**

- Tourisme et Environnement (TEE) ;
- Tourisme et Hôtellerie (TEH).

Ces filières sont rattachées aux besoins du marché tant national qu'international du travail ; garantissant la formation de jeunes efficaces et prêts à affronter une vie professionnelle à tous les niveaux.

I.5. Performances et prix obtenus par l'ISPM

Tableau n° 2: Performances et prix obtenus par ISPM

RANG	PROJET/CONCOURS	ANNEE	ORGANISATEUR
1 ^{er}	Logiciel de Fanorona	1997	Ambassade de France
3 ^e	Repiqueuse SRI	2003	FIER Mada
3 ^e	Tracteur et repiqueuse	2003	Forum de la recherche MENRS
1 ^{er}	Analyse de la structure Biotechnologique du riz	2006	Salon de la créativité et de l'invention Université d'Antananarivo et MENRS
2 ^e	Etuve phytotronique assisté par ordinateur	2006	Salon de la créativité et de l'invention Université D'Antananarivo et MENRS
1 ^{er}	Réchaud à éthanol	2007	Fondation TanyMeva
1 ^{er}	Valorisation de la jacinthe d'eau pour la fertilisation en aquaculture	2008	Fondation TanyMeva
1 ^{er}	Production d'un plastique biodégradable	2008	Fondation TanyMeva
1 ^{er}	Smart Plan Managing	2012	Forum de la recherche MENRS
2 ^e	Intelligent House	2012	Forum de la recherche MENRS
4 ^e	Energies renouvelables	2012	Forum de la recherche MENRS
1 ^{er}	Olympiade d'Informatique de Madagascar	2012	Association Madagascar Au Temps TIC (MTT)
2 ^e	Concours de développement d'application Malagasy : « Flash Crédit »	2014	Orange Madagascar
2 ^e	Concours de développement D'application Malagasy : « Safe SMS »	2014	Orange Madagascar

Source : www.ispm-edu.com

I.6. Organigramme de l'ISPM :

Directement rattachée au Rectorat, la direction administrative et financière comporte quatre (4) services : le secrétariat, la surveillance, le professorat et la logistique.

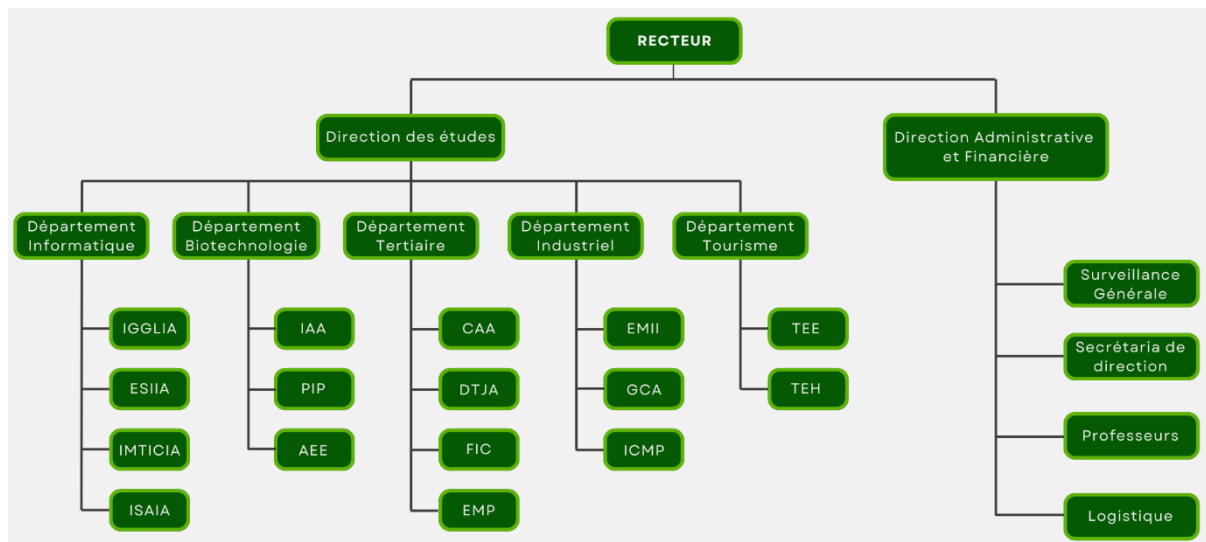


Figure 3: Organigramme de l'ISPM

Source : [15]

Chapitre II : A propos de la Direction au sein du MDAT

II.1. Présentation De L'Observatoire du Territoire (OT)

L'Observatoire du Territoire (créé en 2012) ; selon la loi 2015-051 portant Orientation de l'Aménagement du Territoire, est **chargé de produire et de capitaliser les informations, les connaissances et les réflexions sur l'urbanisme et l'Aménagement du territoire**, afin de fournir des outils d'aide à la décision pour appuyer la formulation et le pilotage des activités d'une part, et évaluer les impacts et l'effectivité de la PNAT d'autre part.

L'OT, faisant partie des directions au sein du Ministère de la Décentralisation et de l'Aménagement du Territoire (MDAT), doit mettre à la disposition des décideurs et des acteurs des informations territoriales crédibles (collecte, capitalisation, traitement, partage et diffusion) sur les dynamiques de l'occupation du sol et la lecture du territoire, et doit également alimenter la réflexion et le débat sur les grands enjeux des politiques nationales.

Dans cette optique, les activités de l'OT se focalisent sur :

- La collecte, saisi et traitement des informations territoriales et en matière d'urbanisme aussi bien qualitative que quantitative pour ensuite les analyser ; et cela d'une manière permanente ou tout au moins régulière dans le temps ;
- La traduction et valorisation de ces informations afin de les mettre à disposition selon certaines formes de rendu qui soient compréhensibles à chacun, et accessibles à travers un Géoportail
- L'évaluation des projets en cours, la capitalisation et diffusion des bonnes pratiques afin d'améliorer la méthodologie d'exécution des projets
- La réalisation des études sur des thématiques bien précises afin d'alimenter une réflexion pour de futures interventions au sein du Ministère
- L'Opérationnalisation de la dématérialisation des autorisations d'urbanisme à travers l'application MATAC

L'Observatoire du Territoire comprend 2 services et 06 personnels, à savoir :

- Le Service d'Analyse et d'Etudes (SAE) ;
- Le Service des Systèmes de la Gestion des Bases de Données (SGBD) ;

La Ministère de l'Aménagement du Territoire se nomme comme suit : « Ministère de la Décentralisation et de l'Aménagement du Territoire », si avant elle était « la Ministère de l'Aménagement du Territoire et Service Foncier ».

Avant les services d'Observatoire y compris l'OAT étaient rattaché au Secrétariat Général, mais à présent ils sont rattachés au Ministre

- **Organigramme du Ministère de la Décentralisation et de l'Aménagement du Territoire :**

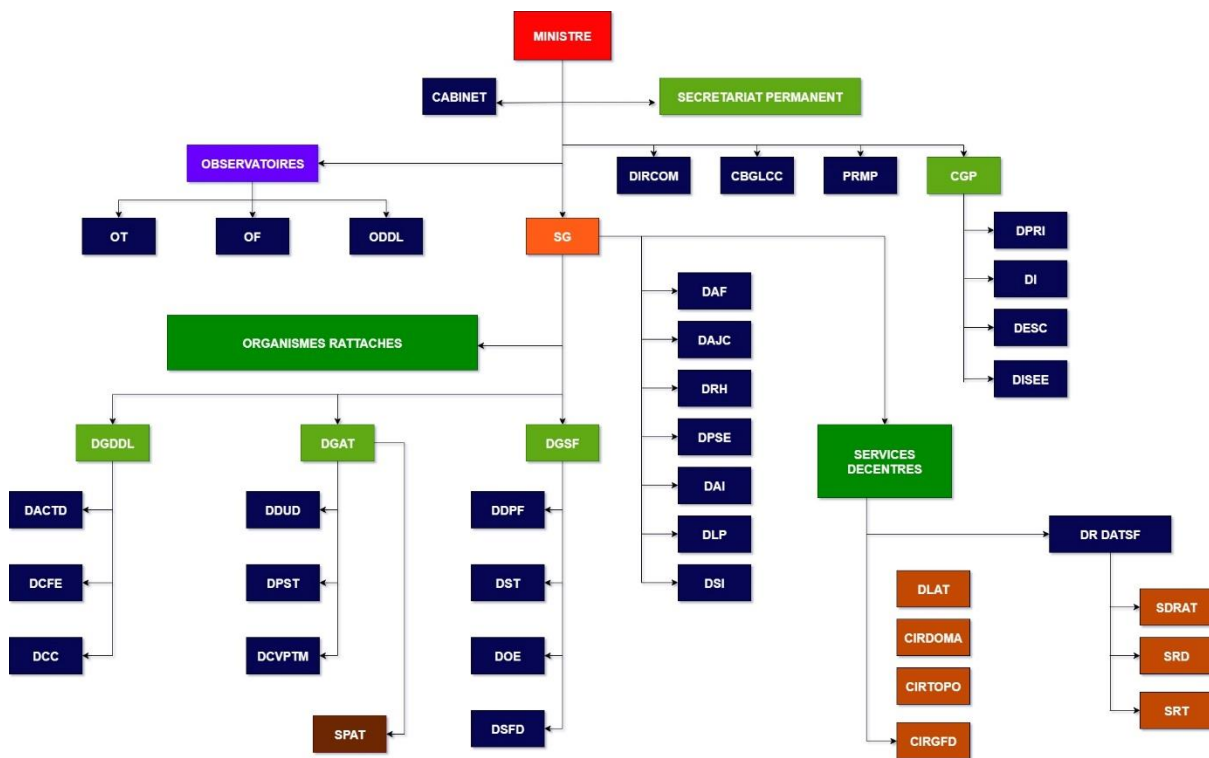


Figure 4: Organigramme du MDAT

Source :[7]

II.2. Les missions

II.2.1. Les missions des Observatoires :

Ses missions concernent surtout à :

- La production, la capitalisation et la valorisation des informations, des connaissances, des réflexions sur leur thématique clé nationaux
- L'analyse de donnée régulières sur le déploiement et le niveau d'opérationnalisation des services du Ministère et des résultats des différents programmes
- La réalisation des études thématiques sur les problématiques clés et l'amélioration du gestion du secteur étudié
- Et enfin, la mise diffusion et la mise en débat des informations produites

II.2.2. Les missions de l'OT :

L'Observatoire du Territoire est notamment chargé de :

- Analyser les modes de gestion territoriale et de formuler les observatoires y afférentes ;
- Tenir des statistiques mise à jour relative à la gestion du territoire à Madagascar
- Élaborer les notes d'aide à la décision pour une meilleur orientation de la gestion territoriale
- Faire un suivi rapproché du respect des divers outil de planification territorial et urbaine : schéma national et schéma régionaux d'aménagement du territoire, schéma d'aménagement communaux, plan d'urbanisme directeur et de détails
- Assurer la veille informationnelle en matière d'aménagement du territoire ;
- Veiller à opérationnalisation du système du Ministère en charge de l'Aménagement du territoire : autorisation de construire (MATAC)
- Établir une base de données mises à jour en matière d'urbanisme et d'aménagement du territoire

Chapitre III : Présentation et Choix du Projet

III.1. Contexte et Problématique

La plateforme web du Ministère de la Décentralisation et de l'Aménagement du Territoire joue un rôle clé dans la communication entre le ministère et les citoyens. Cependant, l'ancien site présentait plusieurs problèmes :

III.1.1. Design

Le site n'était pas attractif et ne respectait pas les normes modernes de design sur plusieurs points (prenons exemple sur la page d'accueil):

- Les éléments du site sont trop centrés. Cela laisse des espaces vides sur les côtés.

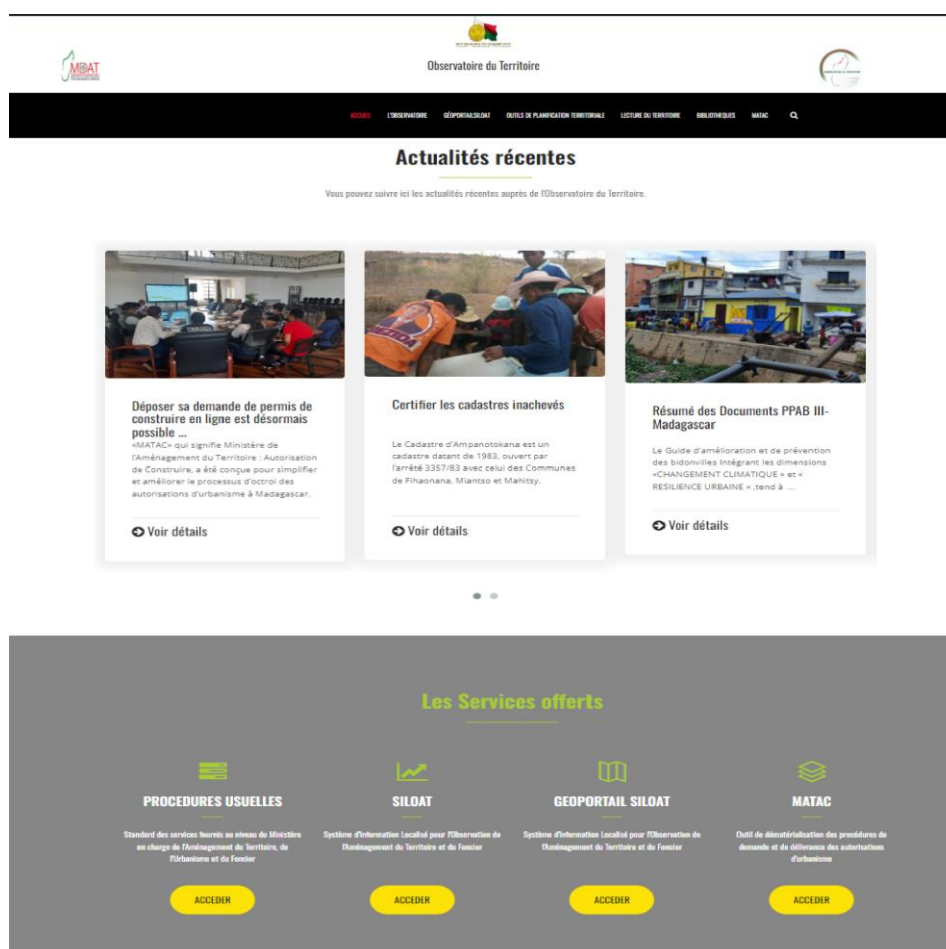


Figure 5: Accueil du site OT (partie 1)

Source : Auteurs

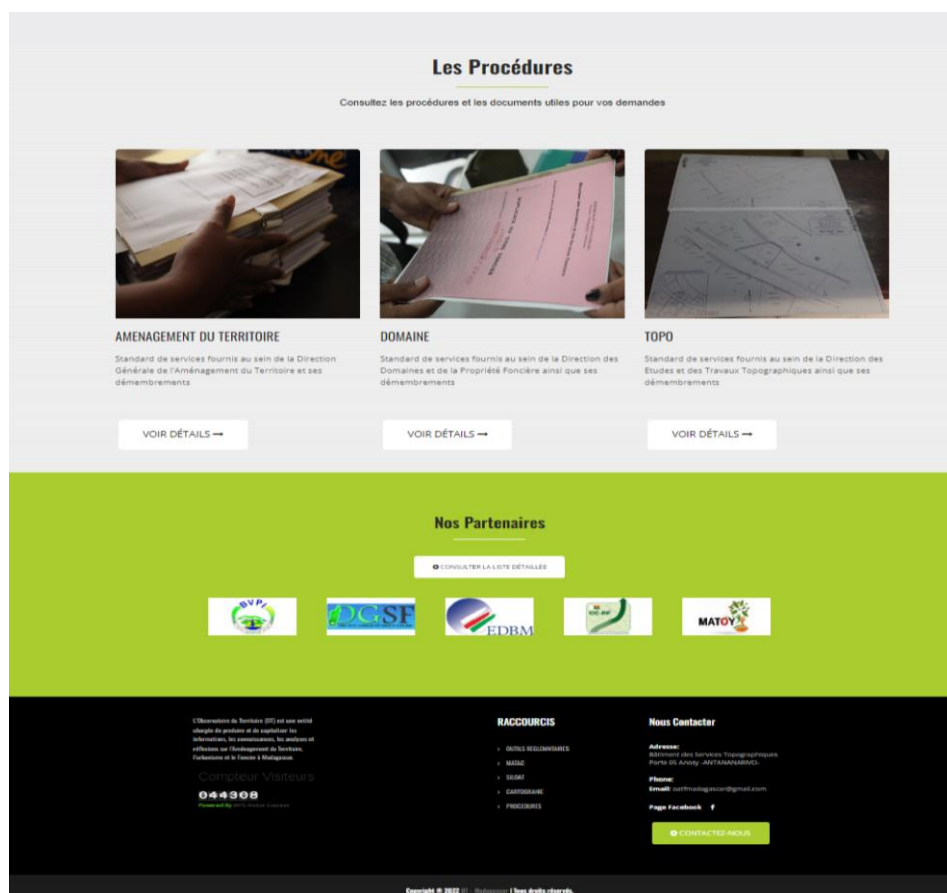


Figure 6: Accueil du site OT (partie 2)

Source : Auteurs

- Les couleurs utilisées sur le site ne sont pas complémentaires, c'est-à-dire que les couleurs ne peuvent pas être utilisées en même temps comme le gris et le jaune.

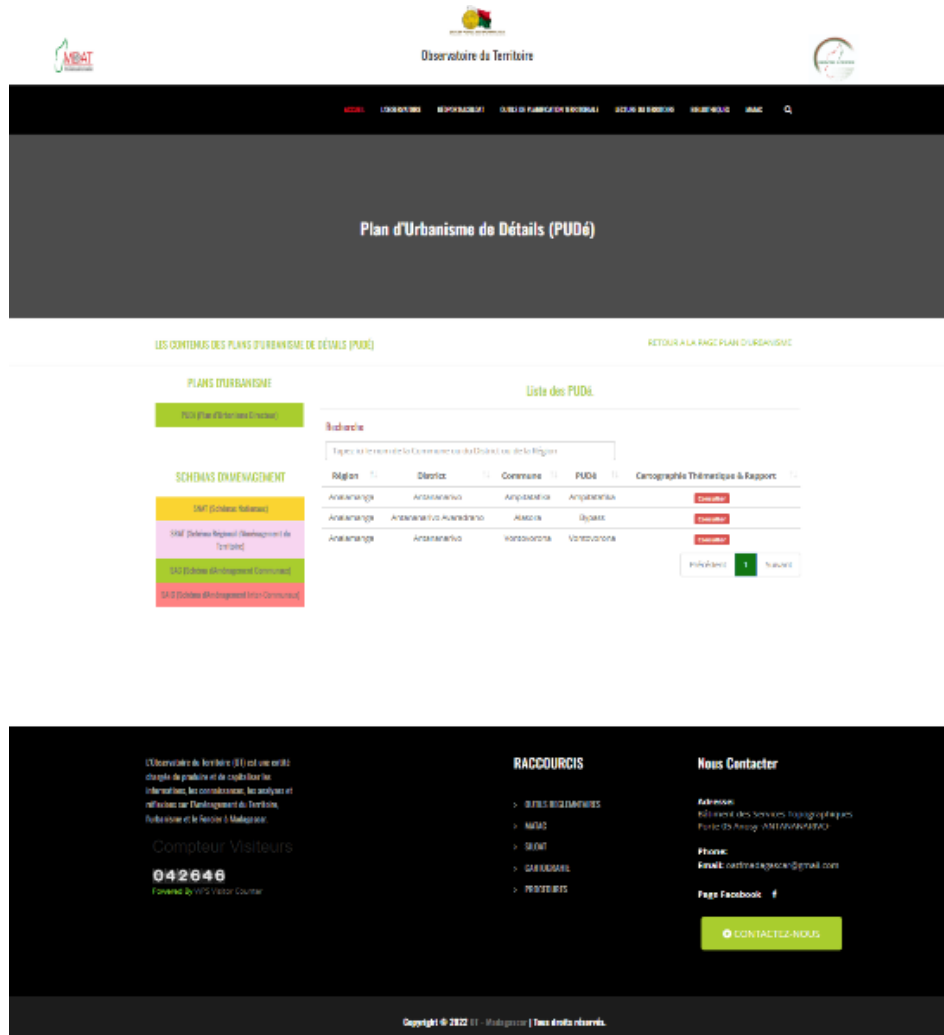


Figure 7 : Extrait de pages du site OT

Source : Auteurs

- Certains éléments sont disposés en désordre et d'autres sont restreints sur de petites zones.



- Pour accéder à un document, on est obligé de passer sur plusieurs pages alors qu'il peut être placé sur une page. Ici par exemple il faut naviguer sur trois pages pour consulter le contenu d'une procédure. Le contenu en question n'est juste que des liens de redirection, que ce soit pour consulter ou pour télécharger.

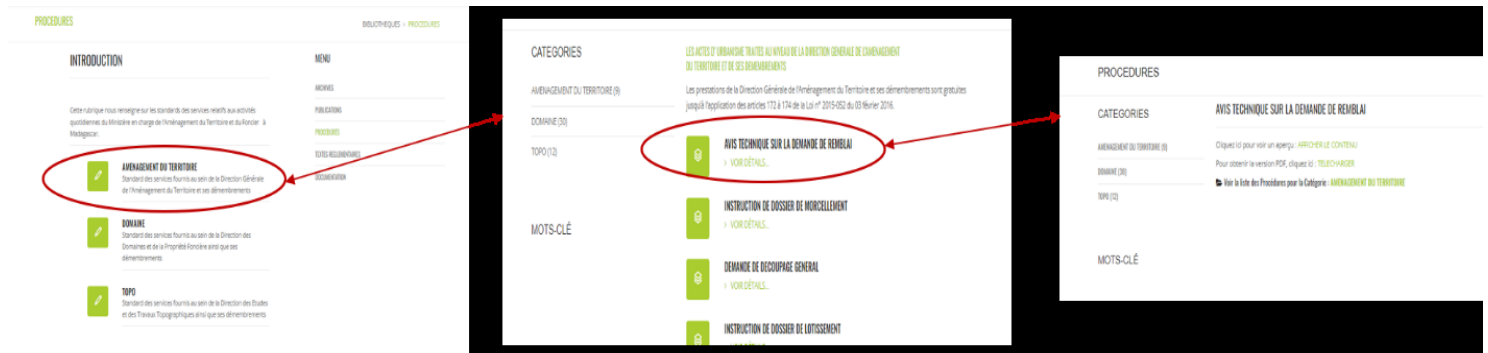


Figure 9 : Navigation du site

Source : Auteurs

III.1.2. Non-adaptabilité :

Certaines pages du site ne s'adaptent pas aux appareils mobiles. Alors, si on accède au site via un téléphone, certains éléments sont mal placés, voire même cachés par d'autres.

III.1.3. Mise à jour manuelle :

Pour inclure des nouvelles données, il était obligé de les insérer dans le "dashboard" du Wordpress pour qu'elles s'affichent à l'interface mais il fallait les insérer une seconde fois dans la base de données. En clair la mise-à-jour devait se faire en deux temps : du côté frontend et ensuite du côté base de données.

III.2. Objectifs du Projet

L'objectif principal de ce projet est de réaliser une refonte complète de la plateforme web afin de répondre aux besoins suivants :

- Modernisation du design : Proposer une interface utilisateur (UI) attrayante et conforme aux normes actuelles.
- Amélioration de l'expérience utilisateur (UX) : Simplifier la navigation et rendre les informations facilement accessibles.
- Compatibilité multi-plateformes : Développer un site responsive accessible depuis les ordinateurs, tablettes et smartphones.

- Facilité de gestion : Mettre en place un espace administrateur convivial permettant une gestion autonome par les employés du ministère.

III.3. Justification du Choix du Projet

Le choix de la refonte de la plateforme web du ministère s'est imposé pour plusieurs raisons :

- La plateforme web est le principal point de contact entre le ministère et les citoyens. Un site fonctionnel et moderne renforce la crédibilité et l'efficacité du ministère.
- Les citoyens s'attendent à une navigation intuitive et à une disponibilité des services en ligne.
- Le projet permettra de tirer partie des technologies récentes, comme les frameworks modernes et les outils d'analyse web.

III.4. Résultats Attendues :

- Une plateforme web moderne, sécurisée et conforme aux normes.
- Une augmentation de la satisfaction des utilisateurs grâce à une meilleure expérience de navigation.
- Une gestion autonome et simplifiée du contenu de navigation.
- Une image renforcée du ministère auprès des citoyens et partenaires.

PARTIE II : CONCEPTION ET DÉVELOPPEMENT

Chapitre IV : Conception du Projet

IV.1. Langage de modélisation unifié (UML)

Afin de mieux reconstruire l'architecture et le fonctionnement de la plateforme web refondue, nous avons eu recours au langage de modélisation unifié (UML).

L'UML est un langage de modélisation graphique conçu pour une méthode normalisée de visualisation dans les domaines du développement logiciel. Dans ce projet, il permet de représenter graphiquement les fonctionnalités du système, sa structure interne, ainsi que les interactions entre les différents composants.

Pour bien visualiser les tâches à faire pour la réalisation de ce projet, nous avons utilisé trois diagrammes UML.

IV.1.1. Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation illustre les différentes interactions possibles entre les utilisateurs et la plateforme. On distingue deux acteurs : l'utilisateur normal et l'administrateur. L'utilisateur peut consulter le contenu de la plateforme, que ce soit les plans d'urbanismes, les publications et autres. L'administrateur est celui qui est responsable du contenu du site. Il gère les comptes admins, insère, modifie, ou supprime des données de la base de données s'il le faut.

Le diagramme ci-dessous met en évidence la séparation des actions et les droits d'accès des acteurs.

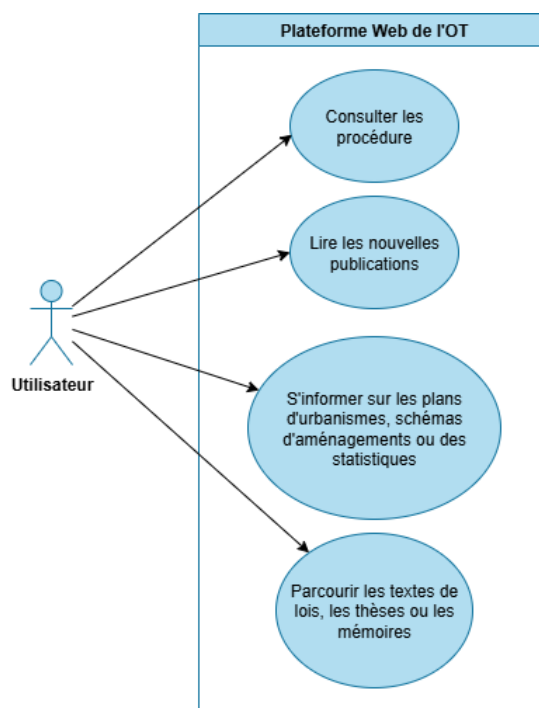


Figure 10 : Diagramme de cas d'utilisation (partie 1)

Source : Auteurs

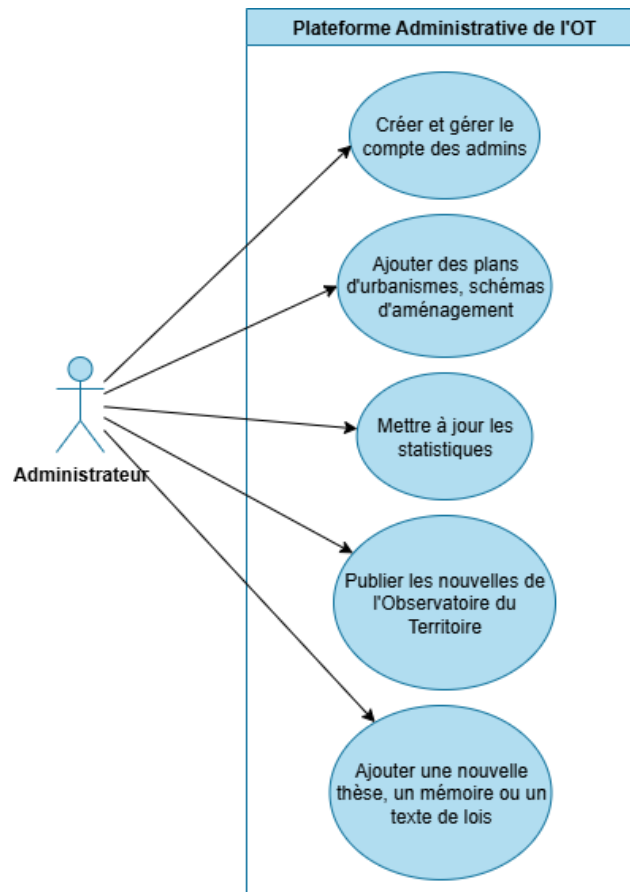


Figure 11 : Diagramme de cas d'utilisation (partie 2)

Source : Auteurs

IV.1.2. Diagramme de Classe

Le diagramme de classe présente la structure logique du système en décrivant les principales entités manipulées par l'application ainsi que leurs attributs. On y retrouve huit classes au final. Ce diagramme met en évidence les relations entre les entités. Ci-dessous, on comprend clairement que c'est la classe Admin qui gère les autres classes.

Ce modèle conceptuel nous a été utile, par la suite, pour la conception de la base de données (PostgreSQL) et pour la structure de code du côté backend (Express JS).

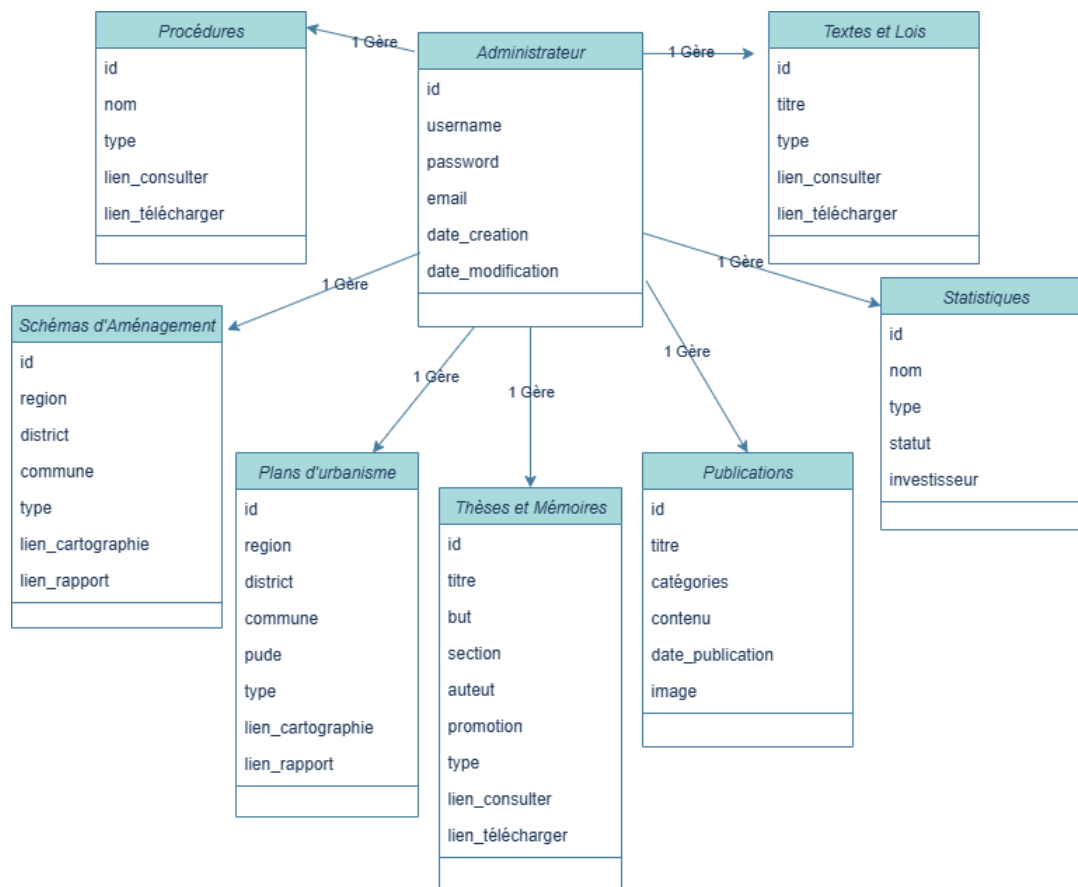


Figure 12 : Diagramme de classe

Source : Auteurs

IV.1.3. Diagramme de Séquence

Le diagramme de séquence illustre le déroulement temporel d'un scénario typique. Dans notre cas, il nous a été utile pour construire la fonction de connexion de l'administrateur. Il décrit pas à pas les échanges entre l'admin, le formulaire, le backend et la base de données.

Ce schéma explique le processus de vérification des identifications, l'utilisation du hashage de mot de passe et la génération de jeton JWT pour accéder à la page admin de la plateforme.

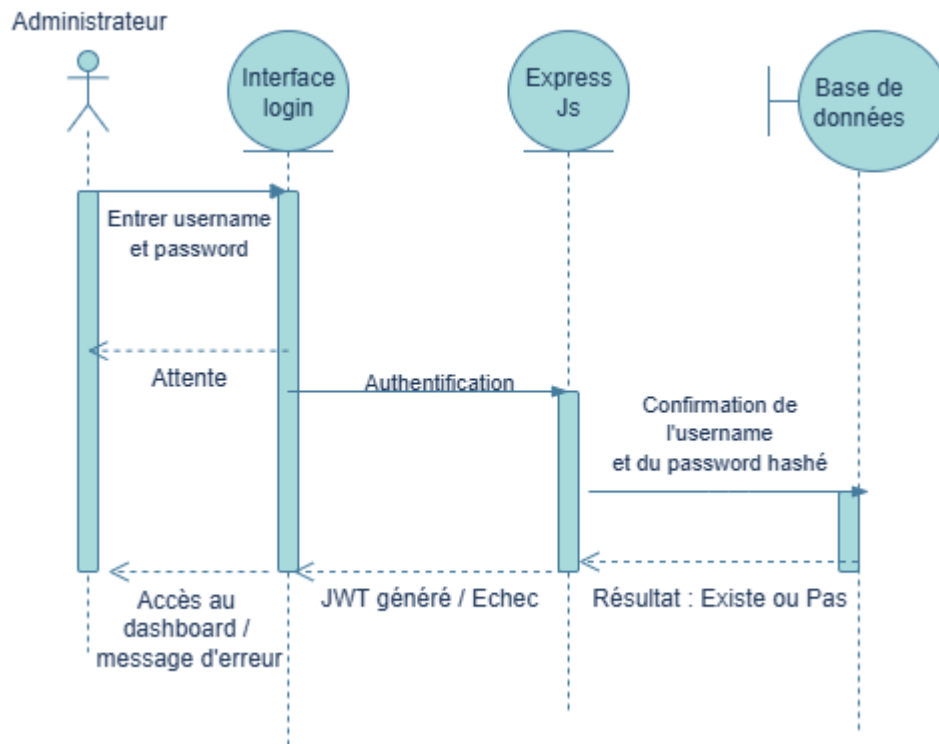


Figure 13 : Diagramme de séquence

Source : Auteurs

IV.2. Conception de l'interface utilisateur (Frontend) [6]

IV.2.1. Objectifs de la refonte frontend

La refonte de la plateforme web de l'Observatoire du Territoire avait pour objectif principal d'améliorer l'expérience utilisateur en modernisant l'interface graphique, en rendant la navigation plus intuitive et en assurant une compatibilité optimale avec les terminaux mobiles (responsive design). L'ancienne version du site, conçue avec des technologies statiques, ne répondait plus aux standards actuels du web, notamment en termes d'accessibilité, de performance et de design adaptatif. Il était donc nécessaire de repenser totalement l'interface utilisateur tout en gardant les fonctionnalités de base de la plateforme.

IV.2.2. Outils et technologies utilisés pour la conception

Pour concevoir les maquettes de la nouvelle interface, l'outil Adobe XD a été utilisé. Ce logiciel permet de créer des maquettes haute fidélité collaboratives, facilement partageables et modifiables. Grâce à sa simplicité d'utilisation, il a été possible de simuler le comportement des pages avant même le début du développement.

Le développement frontend a ensuite été réalisé avec le framework Vue.js, qui permet de construire des interfaces réactives et modulaires en utilisant des composants réutilisables. Le design responsive a été pris en charge par Bootstrap, un framework CSS utilitaire moderne et aussi l'utilisation des 'media query' pour l'adaptabilité à tous les périphériques .

IV.2.3. Principes de design retenus

Plusieurs principes ont guidé la conception visuelle de la nouvelle interface :

- **Simplicité** : L'interface a été épurée afin de faciliter la navigation et l'accès rapide à l'information.
- **Cohérence** : Une palette de couleurs harmonieuse a été définie, inspirée des couleurs officielles de l'administration concernée.
- **Accessibilité** : Une attention particulière a été portée sur le contraste des textes, la taille des boutons et la clarté des libellés.
- **Responsive design** : L'ensemble des pages a été conçu pour s'adapter automatiquement aux différentes tailles d'écran (ordinateur, tablette, smartphone).

IV.2.4. Extraits des maquettes

IV.2.4.1. Maquette de la page d'accueil

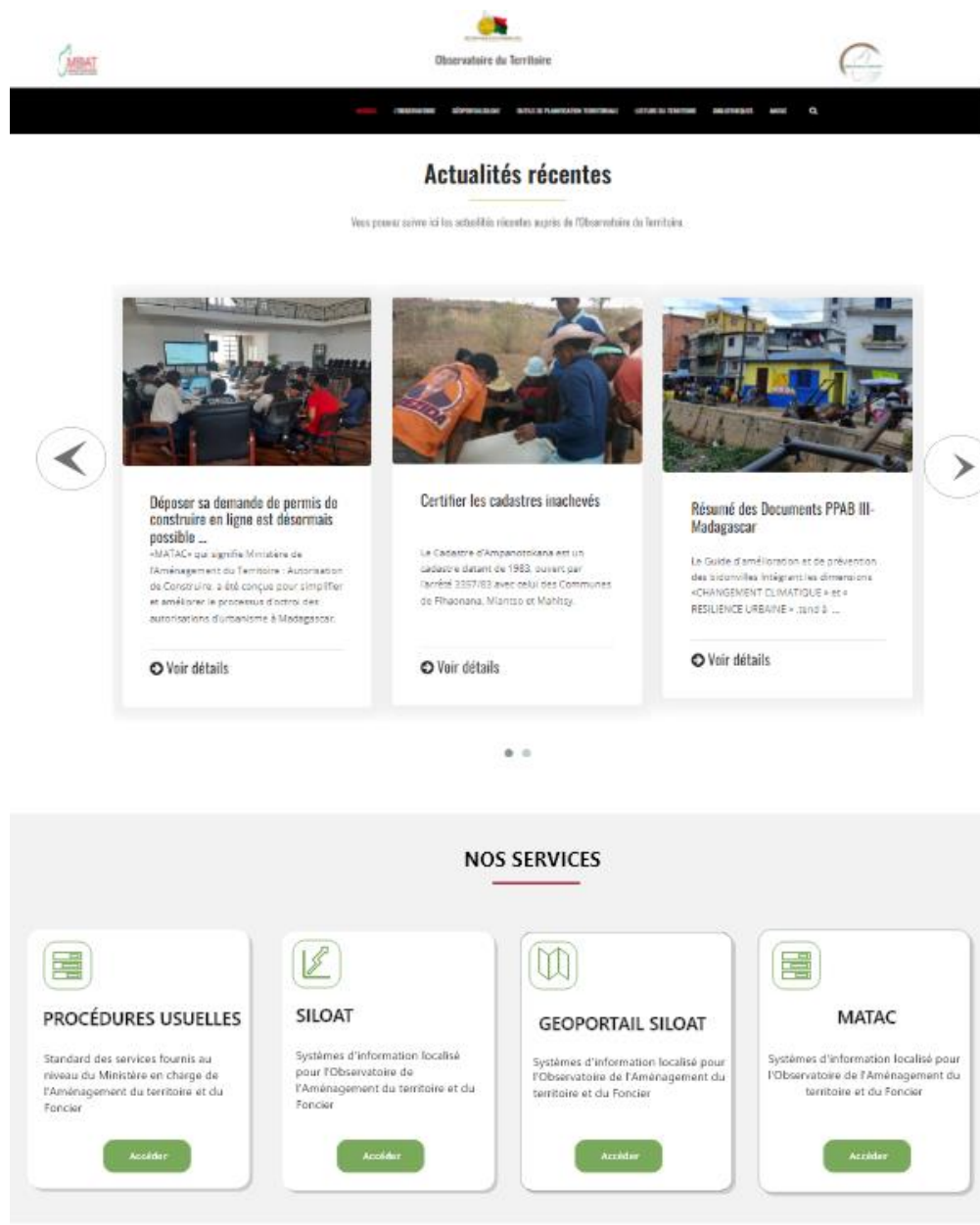


Figure 14 : Maquette de l'accueil

Source : Auteurs

La page d'accueil a été pensée comme un point d'entrée simple et clair pour l'utilisateur. Elle comprend un bandeau supérieur avec le logo et le menu principal, une section de présentation de la plateforme, et des accès directs aux outils de planification disponibles. La mise en page utilise une grille à deux colonnes sur desktop, et une disposition verticale sur mobile.

IV.2.4.2. Maquette de la page des outils de planification

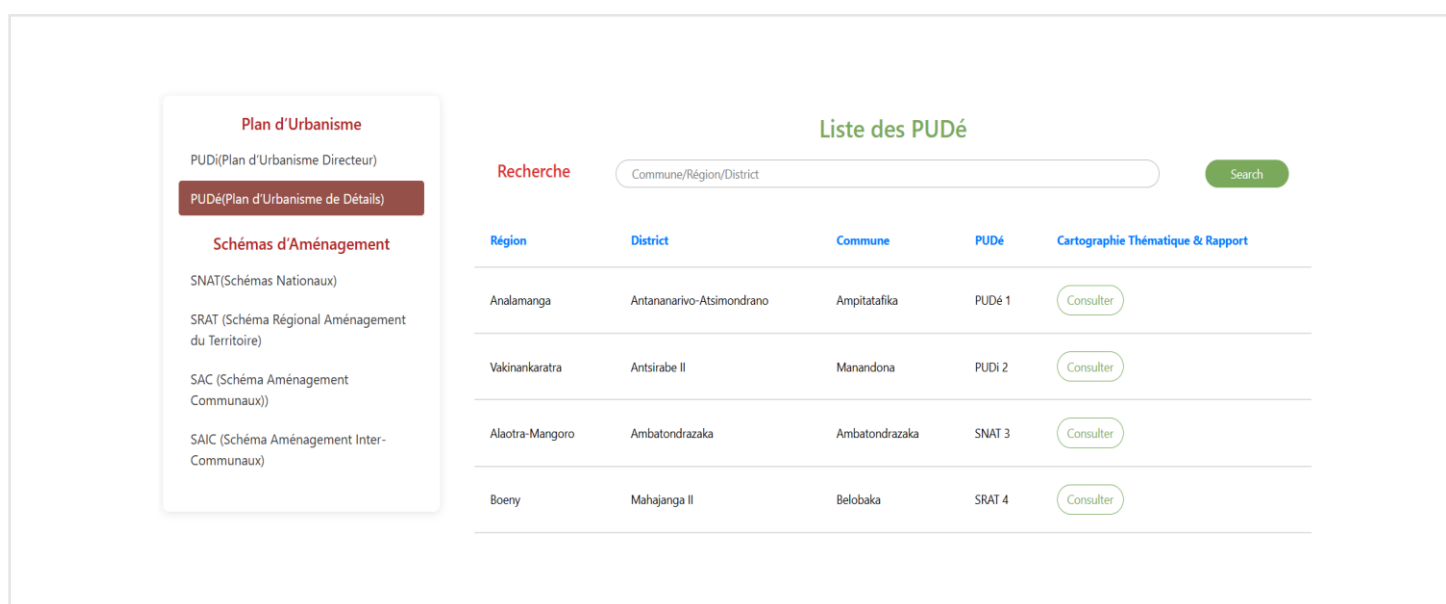


Figure 15 : Maquette de la page des outils

Source : Auteurs

Cette page présente les différents outils sous forme de cartes contenant un titre, une description et un bouton d'accès. Une barre de recherche et des filtres latéraux permettent de trier les outils par catégorie ou par zone géographique. L'utilisation de composants cliquables améliore l'expérience de consultation.

IV.2.4.3. Maquette version mobile (responsive)

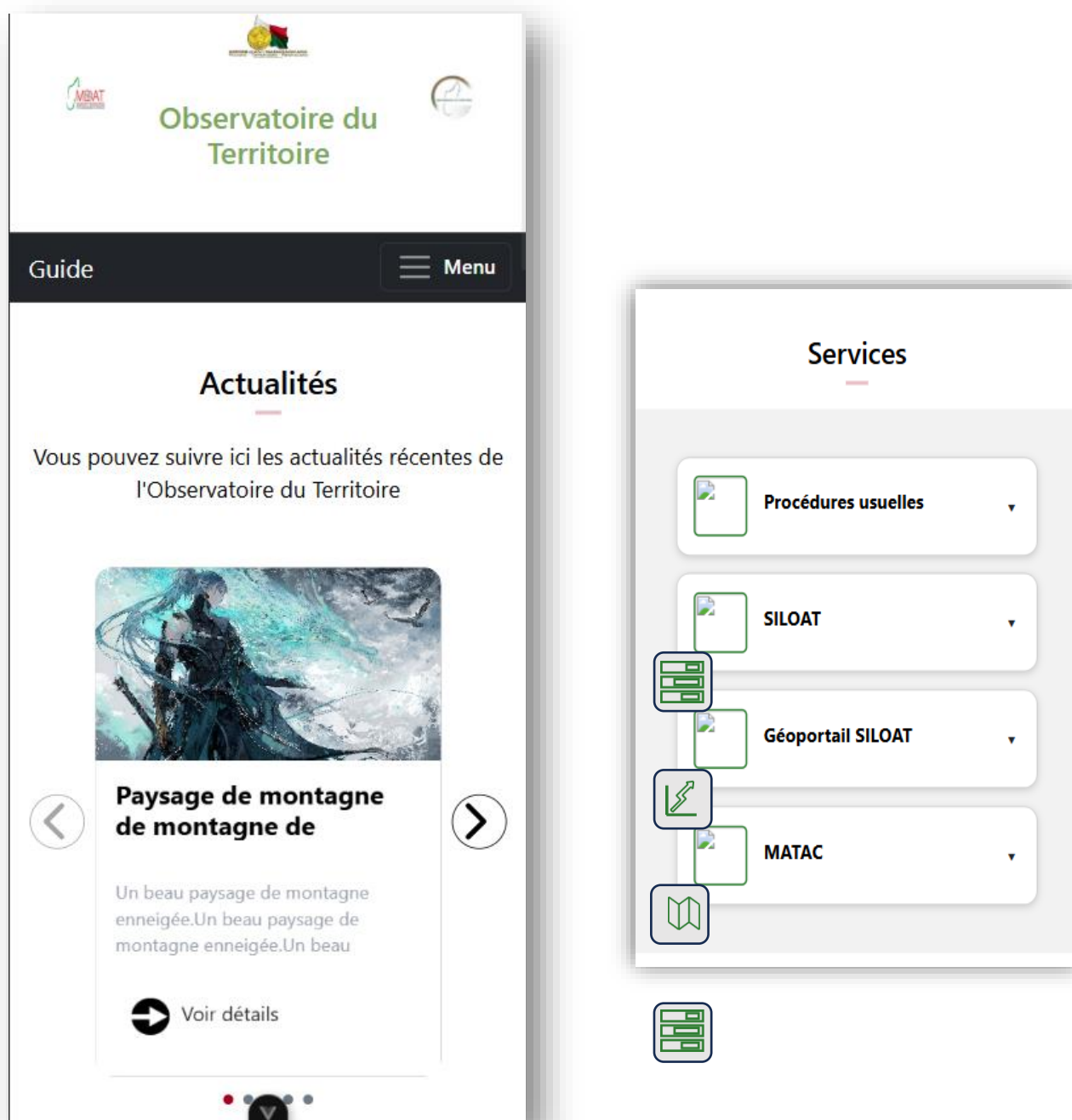


Figure 16 : Maquette de la version mobile de l'accueil

Source : Auteurs

La version mobile conserve les mêmes fonctionnalités mais adopte une disposition verticale pour faciliter la lisibilité sur petits écrans. Le menu de navigation est remplacé par un menu hamburger en haut à droite. Les cartes sont affichées en pleine largeur pour une meilleure visibilité tactile.

IV.2.5. Conclusion de la phase de conception

La phase de conception a permis de poser les bases visuelles et fonctionnelles du frontend avant le codage. Les maquettes ont servi de guide pour le découpage des composants Vue.js, facilitant le développement modulaire et la mise en œuvre du responsive design. Elles ont aussi permis de valider l'ergonomie de l'interface auprès d'utilisateurs testeurs. Cette étape a donc joué un rôle clé dans la réussite de la refonte de la plateforme.

IV.3. Conception du Backend

IV.3.1. Objectifs

Dans le cadre de la refonte, le but est non seulement de concevoir un backend compatible avec la nouvelle interface, mais aussi avec une optimisation des performances, de la maintenabilité, de la sécurité et de l'expérience utilisateur.

Ces étapes étaient cruciales pour ce faire :

Etape 1 : Analyser les limites du site existant (performances, sécurité, ergonomie, structure du code, etc.).

Etape 2 : Reconcevoir le backend :

- Moderniser l'architecture logicielle,
- Faciliter la gestion des routes et des API REST,
- Sécuriser l'accès aux données.

Etape 3 : Refondre la base de données :

- Optimiser les requêtes SQL,
- Assurer l'intégrité et la cohérence des données.

Etape 4 : Mettre en place une API RESTful pour permettre une meilleure communication entre le frontend et le backend.

Etape 5 : Renforcer la sécurité du système par :

- Le hachage des mots de passe,
- L'authentification sécurisée,
- La validation et le filtrage des entrées utilisateurs.

Etape 6 : Optimiser la performance du site en réduisant les temps de réponse du serveur et en améliorant la gestion des requêtes.

Etape 7 : Faciliter l'évolutivité et la maintenabilité du code grâce à une architecture modulaire et bien documentée.

Etape 8 : Améliorer l'expérience utilisateur en assurant une interaction plus fluide et fiable entre l'interface utilisateur et le backend.

IV.3.2. Choix Technologique

Le backend de l'application a été développé en JavaScript à l'aide du framework Express.js, une bibliothèque légère et flexible pour Node.js permettant de créer des serveurs web et des APIs REST. Le choix d'Express.js s'explique par sa simplicité, sa rapidité de mise en œuvre et la richesse de son écosystème.

La base de données utilisée est PostgreSQL, un système de gestion de base de données relationnelle open source robuste, fiable et performant, particulièrement adapté aux applications nécessitant des relations complexes entre les données.

IV.3.3. Rôles

Le backend conçu assure plusieurs fonctions essentielles :

- La gestion des utilisateurs (inscription, connexion, authentification).
- La création, modification et suppression de ressources.
- La connexion et communication avec la base de données PostgreSQL.
- La sécurisation des accès grâce à l'authentification.
- La validation et traitement des données envoyées par le frontend.

IV.3.4. Structure adoptée

Le backend suit une architecture **MVC (Modèle – Vue – Contrôleur)** adaptée à Express :

- **Modèle** : contient la logique de gestion des données et communique avec PostgreSQL.
- **Contrôleur** : contient la logique métier et le traitement des requêtes.
- **Routeur** : relie les requêtes HTTP aux bons contrôleurs.

Chapitre V : Développement du Projet

V.1. Les technologies utilisées

V.1.1. Langage et Framework

V.1.1.1. HTML et CSS [2] [3] :



Figure 17 : Logo HTML

Source : [16]

HTML ou Hyper Text Markup Language est un langage de balisage conçu pour créer la structure de base et le contenu des pages web. Il utilise des balises pour définir les éléments de la page comme les images, les blocs, les tableaux, les titres, les formulaires, etc.... et aussi pour indiquer au navigateur comment afficher le contenu.



Figure 18: Logo CSS

Source : [17]

CSS ou Cascading Style Sheets correspond à un langage informatique permettant de mettre en forme des pages web (HTML ou XML). Ce langage est composé des “feuilles de

style en cascade” appelées fichiers CSS (.css) qui contient des éléments de codage modifiant le style spécifique des éléments comme la taille des polices, la couleur, les marges, les positions, etc....

Le HTML sert de fonction pour toute page web. Il fonctionne en utilisant des balises qui permettent de structurer le contenu. Il est également responsable de l’organisation hiérarchique du contenu qui aide les navigateurs web à interpréter et afficher correctement les éléments. Sans HTML il n’y aurait pas de contenu structuré sur le web. Cependant un page HTML seule serait très basique, c’est-à-dire sans style ni mise en forme. C’est là qu’intervient le CSS, qui permet d’appliquer des règles esthétiques aux éléments HTML et aussi le CSS permet de rendre les sites réactifs qui s’adaptent aux différentes tailles d’écrans. Les fichiers HTML et CSS collaborent pour créer des pages web à la fois fonctionnelles et attractives.

V.1.1.2. Bootstrap [14] :

Bootstrap est un framework CSS et JS open source, créé par twitter, conçue pour faciliter le développement rapide et réactif de sites web. Il fournit une collection de classes CSS et de composants JavaScript prédéfinis, permettant aux développeurs de créer des interfaces utilisateurs modernes et cohérentes sans avoir à écrire beaucoup de codes à personnaliser. Bootstrap inclut des éléments comme les grilles de mise en page, les boutons, les formulaires, les alertes, les modales et bien plus encore, tous optimisés pour être compatible avec une large gamme de navigateurs et de dispositifs.



Figure 19: Logo Bootstrap

Source : [18]

Caractéristiques de Bootstrap :

- **Responsive Design** : Bootstrap repose sur un système de grille (grid system) qui permet de concevoir des interfaces adaptées à tous les types d'écrans (ordinateurs, tablettes, smartphone).
- **Pré-conçus** : Il fournit une grande variété de composants prêts à l'emploi, comme des boutons, des formulaires, des menus de navigation, des alertes, des carrousels, etc.
- **Personnalisable** : Les développeurs peuvent modifier les styles par défauts en utilisant des fichiers CSS personnalisés ou en modifiant les variables SCSS intégrées.
- **Support des framework JavaScript** : Bootstrap inclut des fonctionnalités interactives basées sur JavaScript comme des modales, des menus déroulants, des onglets et des pop overs.

V.1.1.3. JavaScript [4] :



Figure 20: Logo JavaScript

Source : [19]

JavaScript est un langage de programmation utilisé principalement pour ajouter des fonctionnalités dynamiques et interactives aux pages web. Contrairement au HTML qui structure le contenu, et au CSS qui le stylise, le JavaScript permet d'agir sur le comportement des éléments. Par exemple, il peut être utilisé pour créer des animations, valider des formulaires, afficher des alertes, ou encore mettre à jour le contenu d'une page sans avoir besoin de la recharger. Ce langage est exécuté directement dans le navigateur de l'utilisateur, ce qui le rend rapide et réactif. De plus, il est largement compatible avec d'autres technologies comme les API et les frameworks, ce qui en fait un outil essentiel pour le développement web moderne. Grâce à JavaScript, les sites web passent d'un simple affichage statique à une expérience interactive et engageante.

V.1.1.4. Node.JS

Node.JS est une plateforme open source, multiplateforme qui permet d'exécuter du code JavaScript côté serveur, créé en 2009, il repose sur le moteur V8 de Google Chrome, ce qui lui permet d'être rapide et performant. Avec Node.JS, il est possible de développer des applications web, des API, des serveurs, et bien d'autres types de services "backend".



Figure 21: Logo Node.JS

Source : [20]

Caractéristiques de Node.JS :

- Asynchrone et non-bloquant : Toutes les opérations d'E/S(lecture/écriture dans les fichiers, requêtes HTTP, etc.) sont gérées de manière asynchrone pour une meilleure performance.
- NPM (Node Package Manager) : Une bibliothèque contenant des milliers de modules (packages) prêts à l'emploi pour accélérer le développement.
- Léger et rapide : Basé sur le moteur V8, il optimise la vitesse d'exécution du JavaScript.
- Idéal pour les applications temps réel : Grâce à son modèle événementiel, Node.JS est parfait pour des applications comme les chats, les jeux en ligne ou encore le streaming.

V.1.1.5. Vue.JS [14]

Vue.JS est un framework JS progressif utilisé pour construire des interfaces utilisateurs (UI) et des applications web. Créé par Evan You il est devenu l'un des frameworks les plus populaires grâce à sa simplicité, sa flexibilité et ses performances.



Figure 22: Logo VueJS

Source : [21]

Caractéristiques :

- Vue.JS repose sur un modèle basé sur des composants. Chaque composant est une petite partie d'interface utilisateur (UI) réutilisable et modulaire.
- Les composants permettent de structurer une application en petites parties indépendantes.
- Le cœur de Vue.JS repose sur un système de réactivité. Lorsque les données changent, l'UI est automatiquement mis à jour, sans besoin de manipuler directement le DOM.
- Vue.JS peut être facilement intégré dans des projets existants ou utilisé conjointement avec d'autres bibliothèques ou frameworks.

V.1.1.6. Express.JS

Express.JS est un framework web minimaliste et flexible pour Node .JS, conçu pour simplifier la création d'applications web ou API. Il est largement utilisé grâce à sa rapidité, sa légèreté et sa capacité à s'intégrer facilement avec d'autres bibliothèques ou outils.



Figure 23: Logo ExpressJS

Source : [22]

Caractéristiques :

- Express.JS n'impose pas de structure rigide.
- Il peut être personnalisé avec des middlewares pour ajouter des fonctionnalités spécifiques.
- Express.JS est largement utilisé pour créer des API “backend” qui communiquent avec des clients “frontend” comme Angular, React ou Vue.JS
- Il peut aussi gérer des applications web dynamiques grâce à sa compatibilité avec les moteurs de “templates”.

V.1.1.7. SQL [5] :

SQL (Structured Query Language, ou langage de requêtes structurée) est un langage standard utilisé pour interagir avec des bases de données relationnelles. Il permet de manipuler et de gérer des données stockées dans des tables, ainsi que de structurer la base de données elle-même.



Figure 24: Logo SQL

Source : [23]

Objectifs :

- Les manipulations des données comme ajout, modification, suppression, consultation.
- La gestion de la structure : créer, modifier et supprimer des tables ou d'autres objets dans une base de données.
- Sécurité : définir qui peut accéder au contenu de la base de données.

V.1.1.8. PostgreSQL**Figure 25: Logo PostgreSQL****Source : [24]**

PostgreSQL souvent abrégé en PostgreS est un système de gestion de base de données relationnelle (SGBDR) open source puissant, fiable, et hautement extensible. Conçu pour être robuste et performant, il est largement utilisé pour gérer des données complexes dans des environnements variés, allant des petites applications aux grandes entreprises.

Caractéristiques :

- PostgreS est un SGBDR avec des fonctionnalités orientées objet. Il permet de définir des types de données personnalisées, des fonctions et mêmes des objets héritables.
- Il supporte plusieurs langages de programmation pour les fonctions (PL/pgSQL, Python, etc)
- Il peut traiter des documents JSON comme des bases de données NoSQL, en plus de son modèle relationnel classique.
- Gestion fine des permissions utilisateurs (GRANT/REVOKE).
- Support natif du chiffrement SSL pour sécuriser les connexions.

V.1.1.9. Syntactically Awesome StyleSheets

SASS ou Syntactically Awesome StyleSheets est un préprocesseur CSS qui ajoute des fonctionnalités avancées au langage CSS standard. Il permet d'écrire du code plus structuré, maintenable et efficace en introduisant des concepts comme les variables, les fonctions et l'imbrication de style.

Fonctionnalités:

- Variables : permet de stocker des valeurs (couleurs, taille, police) et de les réutiliser dans tout le code.
- Imbrication des styles : évite la répétition en imbriquant les règles CSS à l'intérieur de leurs parents.
- Partiales et imports : permet de diviser le code en plusieurs fichiers et de les importer dans un fichier principal.

V.1.2. Logiciels et Extensions

V.1.2.1. Visual Studio Code

Visual Studio Code (VS Code) est un éditeur de code open source, gratuit, développé par Microsoft. Il est largement utilisé par les développeurs pour écrire, éditer et déboguer du code dans plusieurs langages de programmation. Sa popularité repose sur sa légèreté, sa flexibilité et ses puissantes fonctionnalités.



Figure 26: Logo VS Code

Source : [25]

Caractéristiques :

- VS Code est conçu pour être rapide à démarrer. Même sur des machines moins puissantes, tout en offrant des fonctionnalités avancées.
- Il prend en charge une multitude de langages tels que JS, Python, Java, C++, PHP, HTML, CSS, Go, Ruby, etc.
- VS Code dispose d'une vaste bibliothèque d'extensions disponible via le Marketplace.
- Les extensions permettent d'ajouter des fonctionnalités comme l'intégration de GIT, le support des frameworks, ou des outils spécifiques à un langage.
- Il fournit des suggestions intelligentes basées sur le contexte appelé "IntelliSense".
- Il supporte la complétion de code, la correction d'erreurs et la suggestion pour les imports.
- Il permet de déboguer directement depuis l'éditeur avec des points d'arrêts, l'inspection des variables, et des consoles interactives.
- Il permet d'accéder rapidement au terminal sans quitter l'éditeur (Bash, PowerShell, Command Prompt, etc).

V.1.2.2. Adobe XD [6]

Adobe XD est un logiciel de conception et de prototypage d'UX et d'UI développé par Adobe, conçu pour répondre aux besoins des designers d'applications et des sites web.



Figure 27: Logo Adobe XD

Source : [26]

Fonctionnalités d'Adobe XD

- Conception vectorielle : Permet de créer des interfaces avec des outils de dessin vectoriel, des grilles et des composants réutilisables
- Prototypage interactif : ajout des liens entre les écrans pour simuler la navigation et tester l'expérience utilisateur
- Système de design : gère des bibliothèques de composants et de style pour assurer la cohérence des designs

V.1.2.3. pgAdmin 4

pgAdmin4 est un outil open source de gestion et d'administration de base de données PostgreSQL. Il fournit une interface graphique permettant aux utilisateurs d'interagir facilement avec une base de données, d'exécuter des requêtes SQL et de gérer les objets de la base de données.

Fonctionnalités de pgAdmin4

- Interface graphique intuitive : permet de naviguer facilement entre les bases de données, tables, schémas et autres objets.
- Editeur SQL avancé : fournit une coloration syntaxique, l'auto-complétions et des outils de débogages pour exécuter et optimiser les requêtes SQL.
- Gestion des utilisateurs et rôles : permet d'ajouter, modifier et gérer les autorisations des utilisateurs et groupes.
- Explorateur d'objets : Affiche une vue structurée des bases de données PostgreSQL, y compris les tables, vues, procédures stockées, index et déclencheurs.
- Outil d'analyse de reporting : offre des fonctionnalités pour visualiser les performances des requêtes et surveiller l'état du serveur PostgreSQL.
- Sauvegarde et restauration : permet d'effectuer des sauvegardes et des restaurations des bases de données en quelques clics.
- Support multiplateforme : compatible avec Windows, MacOS et Linux.

V.1.2.4. Vetur



Figure 28: Logo Vetur

Source : [27]

Vetur est une extension pour Visual Studio Code qui fournit un support complet pour le développement avec Vue.JS. Elle offre des fonctionnalités avancées pour améliorer la productivité des développeurs travaillant sur des projets Vue.JS.

Fonctionnalités:

- Coloration syntaxique et auto-complétions:
 1. Améliore la lisibilité du code Vue.
 2. Fournit des suggestions intelligentes pour JavaScript, TypeScript, CSS et HTML dans les fichiers '.vue'.
- Linting et formatage automatique:
 1. Intègre ESLint et Prettier pour vérifier et formater le code.
 2. Permet d'appliquer des règles de style cohérente dans le fichier Vue.
- IntelliSense avancé:
 1. Fournit des suggestions de code pour les composants Vue, les props et les directives.
 2. Fonctionne avec TypeScript pour améliorer la détection des erreurs.

V.2. Structure de la Base de Donnée et Extraits de Codes

V.2.1. Structure de la Base de Données

Suite à notre analyse du site et au diagramme de classe mis en place, nous avons mis en place une base de données correspondant à ces nouveaux besoins. La base de données a été construite avec PostgreSQL. Bien qu'il s'agisse d'un SGBD relationnel, la structure adoptée est non-relationnelle : la plupart des tables fonctionnent de manière indépendante, sans clés étrangères ni jointures explicites. Ce choix a été motivé par le besoin de modularité, de simplicité de maintenance et de réduction des dépendances entre modules fonctionnels.

Précisons, en premier lieu, que pour toutes les tables sauf la table *admins*, la colonne « identifiant » est du type UUID. C'est une chaîne unique de 128 bits utilisée pour identifier de manière sûre un objet, une ligne ou une entité, sans se baser sur des séquences auto-incrémentées.

Par défaut, l'identifiant est généré par *uuid_generate_v4()*. Il génère un UUID aléatoire mais unique.

Notre base de données ici comprend huit (8) tables :

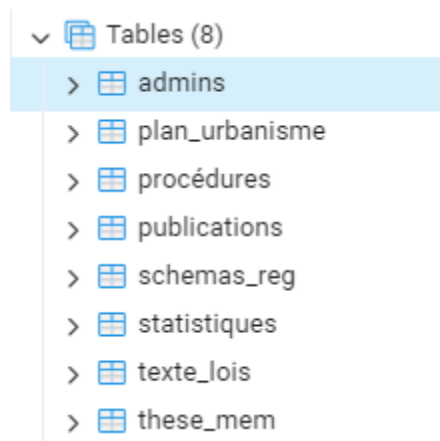
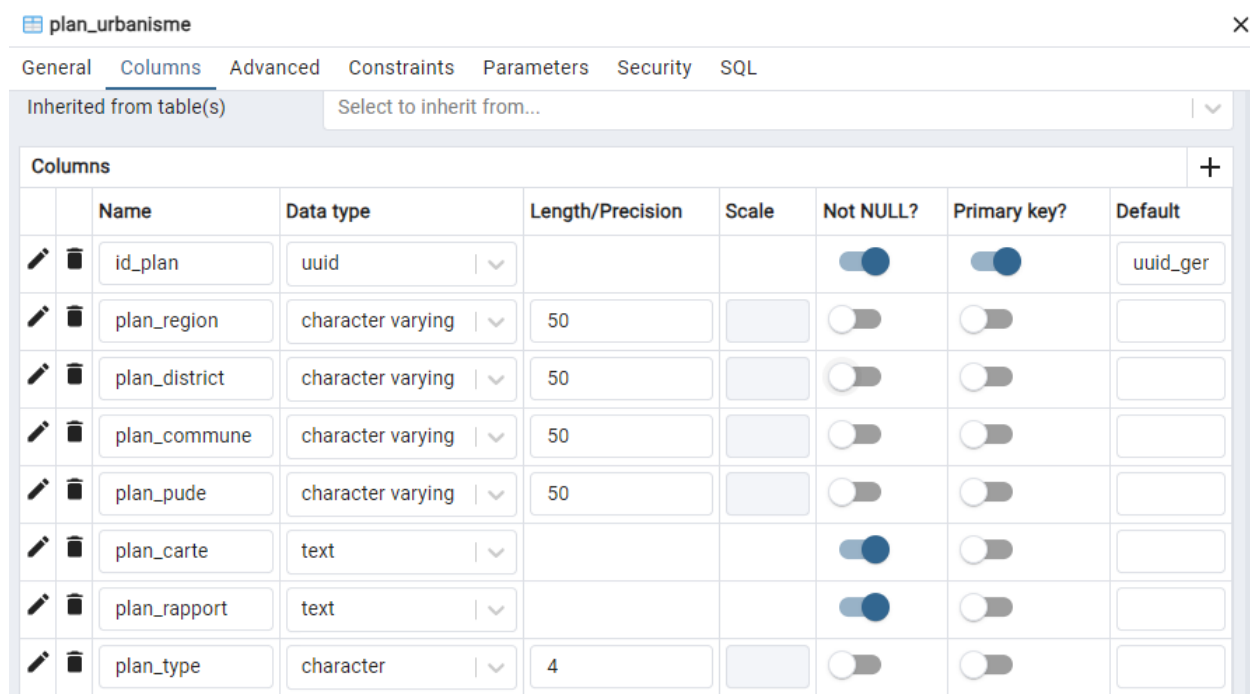


Figure 29: Liste des Tables

Source :Auteurs

V.2.1.1. Table plan urbanisme

Elle contient les données sur les planifications du développement urbain (plan d'urbanisation) de l'OT.



Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
id_plan	uuid			☒	☒	uuid_generate_v4()
plan_region	character varying	50		☐	☐	
plan_district	character varying	50		☐	☐	
plan_commune	character varying	50		☐	☐	
plan_pude	character varying	50		☐	☐	
plan_carte	text			☒	☐	
plan_rapport	text			☒	☐	
plan_type	character	4		☐	☐	

Figure 30: Champs de la table plan_urbanisme

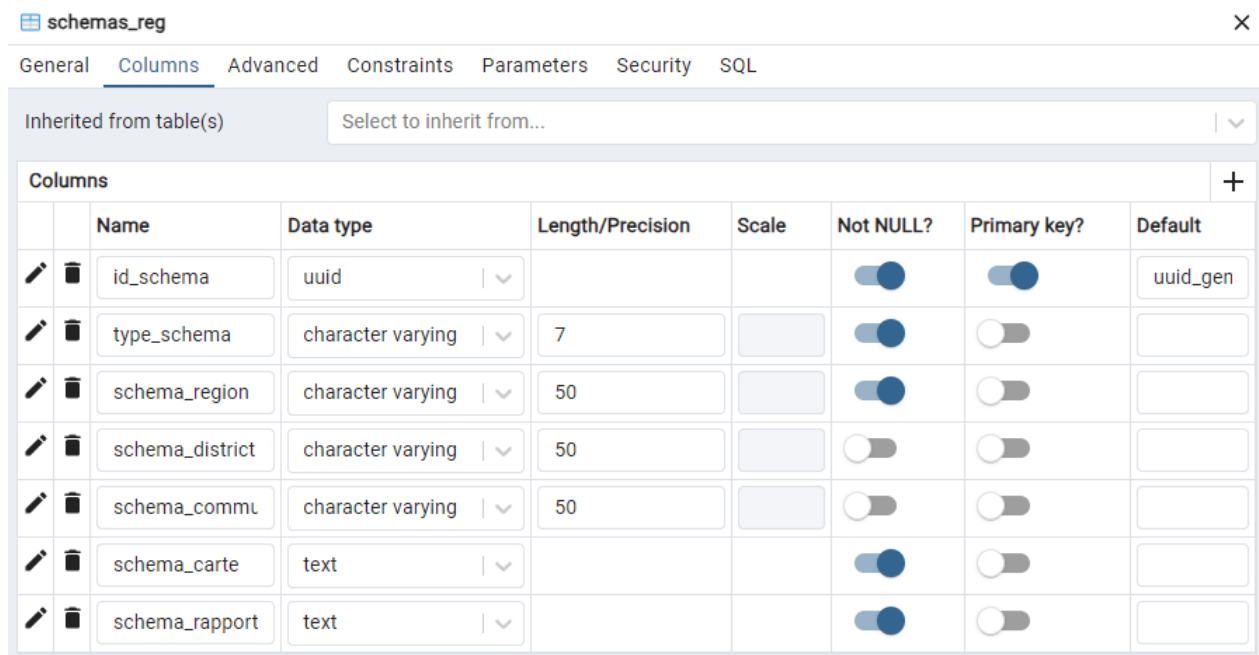
Source :Auteurs

- Champ : plan_region
- Type : liste de caractères à 50 maximum
- Description : La région où un plan d'urbanisme s'applique.
- Champ: plan_district
- Type : liste de caractères à 50 maximum
- Description : La région où un plan d'urbanisme s'applique.
- Champ: plan_commune
- Type : liste de caractères à 50 maximum
- Description : La commune où un plan d'urbanisme s'applique
- Champ: plan_pude
- Type : liste de caractères à 50 maximum
- Description : Plan d'Urbanisme de Détail de la commune
- Champ: plan_carte
- Type: texte
- Description : L'URL de la carte de la commune

- Champ: plan_rapport
- Type: texte
- Description : L'URL du rapport du Plan d'Urbanisme
- Champ: plan_type
- Type: chaîne à 4 caractères
- Description : Le type de plan d'urbanisme. (PUDé ou PUDI)

V.2.1.2. Table schema_reg

Elle stocke les données sur les planifications spatiales qui définit les grandes orientations d'organisation du territoire, à l'échelle d'une région, d'une intercommunalité ou d'une grande ville.



schemas_reg							
General Columns Advanced Constraints Parameters Security SQL							
Inherited from table(s) Select to inherit from...							
Columns							
	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	id_schema	uuid			☒	☒	uuid_gen
	type_schema	character varying	7		☒	☐	
	schema_region	character varying	50		☒	☐	
	schema_district	character varying	50		☐	☐	
	schema_commu	character varying	50		☐	☐	
	schema_carte	text			☒	☐	
	schema_rapport	text			☒	☐	

Figure 31: Champs de la table schemas_reg

Source : Auteurs

- Champ : type_schema
- Type : liste de caractères à 7 maximum
- Description : Le type de schéma d'aménagement
- Champ: schema_region
- Type : liste de caractères à 50 maximum
- Description : La région où un schéma d'aménagement s'applique.

- Champ: schema_district
- Type : liste de caractères à 50 maximum
- Description : Le district où schéma d'aménagement s'applique
- Champ: schema_commune
- Type : liste de caractères à 50 maximum
- Description : La commune où un schéma d'aménagement s'applique
- Champ: schema_carte
- Type: texte
- Description : L'URL de la carte de la commune
- Champ: schema_rapport
- Type: texte
- Description : L'URL du rapport du Schéma d'Aménagement

V.2.1.3. Table procédures

Elle représente ici les standards des services relatifs aux activités quotidiennes du Ministère en charge de la Décentralisation et de l'Aménagement du Territoire à Madagascar.

procédures

General

Columns

Advanced

Constraints

Parameters

Security

SQL

Inherited from table(s)

Select to inherit from...

Columns

		Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
		id_procedure	uuid					uuid_gen
		nom_procedure	text					
		type_procedure	character varying	25				
		consult_procedure	text					
		down_procedure	text					

Figure 32: Champs de la table procédures

Source : Auteurs

- Champ: nom_procedure
- Type : texte
- Description : Nom de la procédure

- Champ: type_procedure
- Type : liste de caractères à 25 maximum
- Description : Type de la procédure (Aménagement du Territoire, Domaine ou Topo)
- Champ: consult_procedure
- Type: texte
- Description : L'URL du document à consulter
- Champ: down_procedure
- Type: texte
- Description : L'URL pour télécharger le document

V.2.1.4. Table publications

Elle contient les publications à propos des travaux du Ministère.

publications

General

Columns

Advanced

Constraints

Parameters

Security

SQL

Inherited from table(s)

Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	id_pub	uuid					uuid_gen
	titre_pub	text					
	contenu_pub	text					
	date_pub	date					
	cat_pub	text[]					
	img_pub	text					

Figure 33: Champs de la table publications

Source : Auteurs

- Champ: titre_pub
- Type : texte
- Description : Titre de la publication
- Champ: contenu_pub
- Type : texte
- Description : Le développement de la publication
- Champ: date_pub
- Type : date

- Description : La date quand la publication a été faite
- Champ: cat_pub
- Type: Tableau de texte
- Description : Les catégories de la publication
- Champ: img_pub
- Type: texte
- Description : L'URL de l'image d'accompagnement

V.2.1.5. Table stats

Elle représente la liste des outils de planification territoriale élaborée par l'OT.

statistiques

General Columns Advanced Constraints Parameters Security SQL

Inherited from table(s) Select to inherit from...

Columns							
	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
 	id_stats	uuid			☒	☒	uuid_gen
 	nom_stats	character varying	50		☒	☐	
 	type_stats	character varying	20		☒	☐	
 	statut_stats	character varying	40		☒	☐	
 	invest_stats	text			☐	☐	

Figure 34: Champs de la table statistiques

Source : Auteurs

- Champ: nom_stats
- Type : chaîne de caractères à 50 maximum
- Description : Le nom des communes où les outils de planification sont actifs
- Champ: type_stats
- Type : chaîne de caractères à 20 maximum
- Description : Le type d'outil de planification
- Champ: statut_stats
- Type : chaîne de caractères à 40 maximum
- Description : Progression sur le projet (Réalisé, En Cours, En Projet)

- Champ: invest_stats
- Type: texte
- Description : L'investisseur du projet

V.2.1.6. Table these_mem

Elle renferme la liste des thèses et mémoires faites sur les activités du Ministère.

these_mem

GeneralColumnsAdvancedConstraintsParametersSecuritySQL

Columns

		Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
		id_tm	uuid v					uuid_gen
		type_tm	character varying v	7				
		nom_tm	text v					
		but_tm	text v					
		section_tm	character varying v	70				
		auteur_tm	text v					
		promotion	character varying v	50				
		extrait_tm	text v					
		down_tm	text v					

Figure 35: Champs de la table these_mem

Source : Auteurs

- Champ: type_tm
- Type : chaîne de caractères à 7 maximum
- Description : Le type du document (thèse ou mémoire).
- Champ: nom_tm
- Type : texte
- Description : Le nom de la thèse/mémoire
- Champ: but_tm
- Type : texte
- Description : Le but de la thèse/mémoire

- Champ: section_tm
- Type : chaîne de caractères à 70 maximum
- Description : Le domaine développé.
- Champ: auteur_tm
- Type : texte
- Description : L'auteur de la thèse/mémoire
- Champ: promotion
- Type : chaîne de caractères à 50 maximum
- Description : La promotion de l'auteur si c'est un mémoire
- Champ: extrait_tm
- Type : texte
- Description : L'URL pour consulter le document
- Champ: dpwn_tm
- Type: texte
- Description : L'URL pour télécharger le document

V.2.1.7. Table texte_lois

Elle contient les lois sur les activités du Ministère.

texte_lois

General

Columns

Advanced

Constraints

Parameters

Security

SQL

Inherited from table(s)

Select to inherit from...

Columns

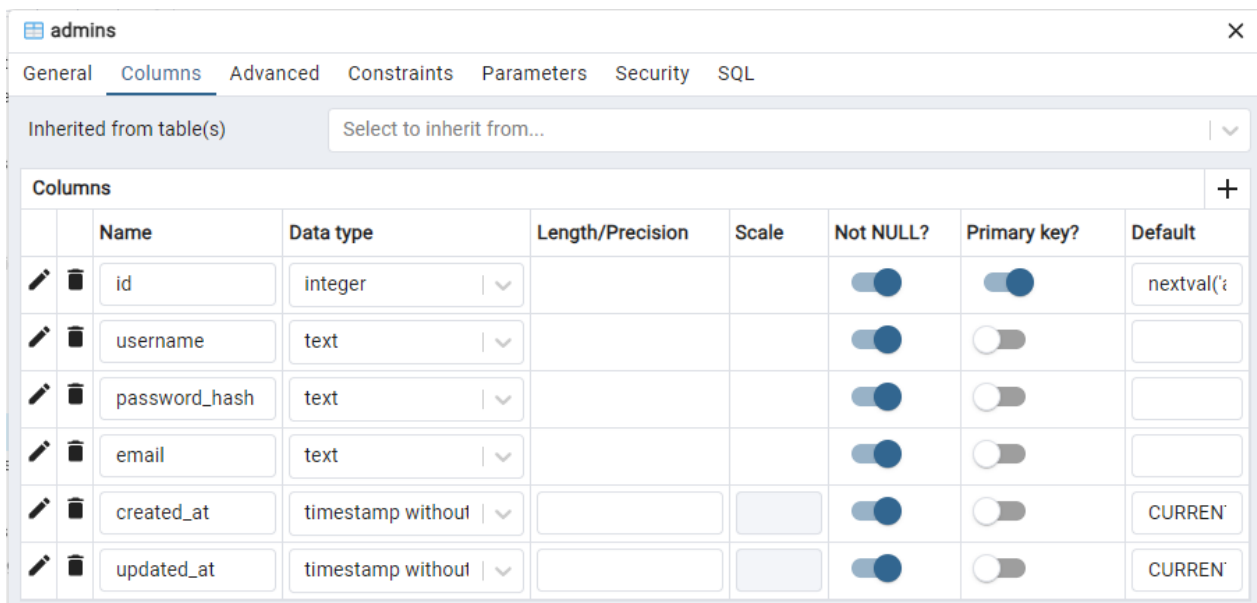
	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
 	id_tl	uuid 					uuid_gen
 	titre_tl	text 					
 	type_tl	text[] 					
 	extrait_tl	text 					
 	down_tl	text 					

Figure 36: Champs de la table texte_lois

Source : Auteurs

- Champ: titre_tl
- Type : texte
- Description : Le titre de la loi
- Champ: type_tl
- Type : tableau de texte
- Description : La catégorie de la loi
- Champ: extrait_tl
- Type : texte
- Description : L'URL pour consulter le document
- Champ: down_tl
- Type: texte
- Description : L'URL pour télécharger le document

V.2.1.8. Table admins



	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
 	id	integer			☒	☒	nextval('i
 	username	text			☒	☐	
 	password_hash	text			☒	☐	
 	email	text			☒	☐	
 	created_at	timestamp without			☒	☐	CURREN'
 	updated_at	timestamp without			☒	☐	CURREN'

Figure 37: Champs de la table admins

Source :Auteurs

- Champ: id.
- Type : entier.
- Description : L'identifiant de la donnée.
- Champ: username.
- Type : texte.
- Description : Pseudonyme pour la connexion.

- Champ: password_hash.
- Type : texte.
- Description : Mot de passe hashé de l'utilisateur.
- Champ: email.
- Type: texte.
- Description : Adresse mail sur lequel le compte est lié.
- Champ: created_at.
- Type : temps.
- Description : Temps à laquelle le compte a été créé.
- Champ: updated_at.
- Type: temps.
- Description : Temps à laquelle les propriétés du compte ont été modifiées.

V.2.2. Extraits de code

V.2.2.1. Back-End

Le backend repose sur une architecture MVC permettant une séparation claire des responsabilités. Il est développé avec Node.js et Express, et interagit avec une base de données PostgreSQL. L'organisation suit une logique modulaire avec trois dossiers principaux : **models**, **controllers**, et **routes**.

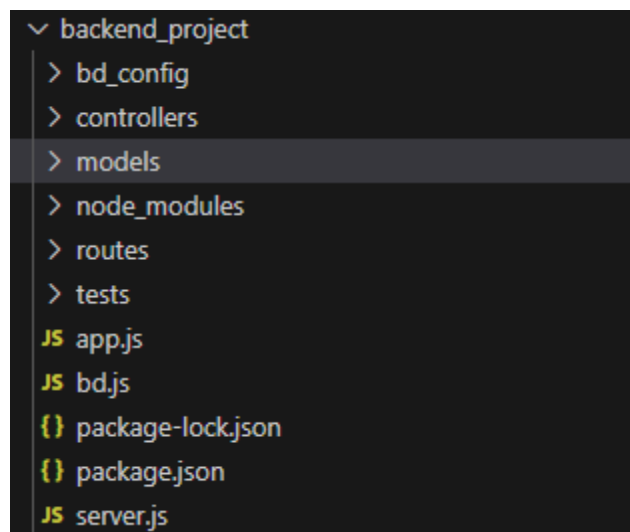


Figure 38: Architecture du backend

Source :Auteurs

L'**architecture MVC** est un **modèle d'organisation de code** très utilisé dans le développement d'applications web.

MVC signifie :

M = Model (Modèle)

V = View (Vue)

C = Controller (Contrôleur)

Ce modèle sépare clairement les responsabilités dans une application.

V.2.2.1.1. Model

Les **models** servent d'interface entre le code backend et la base de données. Ils décrivent la structure des données, même si celle-ci existe déjà dans la base. Grâce aux models, on peut manipuler les données plus facilement, sans avoir à écrire directement des requêtes SQL complexes.

Pour illustrer le fonctionnement de l'architecture MVC mise en place dans le backend, on a choisi la table 'plan_urbanisme'. Elle contient les informations essentielles comme la région, le district, la commune, etc. Le fichier model contient cinq (5) fonctions :

- **Éléments requis:**

```
const bd = require('../bd');
const format = require('pg-format');
```

Figure 39: Imports

Source :Auteurs

__"bd"__ est la liaison entre le fichier model et la base de données.

__"format"__ est la bibliothèque Node.js utilisée avec PostgreSQL pour formater proprement et de manière sécurisée les requêtes SQL dynamiques, notamment des insertions des noms de colonnes ou des valeurs dans une requête.

- **Fonction getAll**

```
//récupérer toutes les données de la table
exports.getAll = async () => {
  const result = await bd.query('SELECT * FROM plan_urbanisme');
  return result.rows;
};
```

Figure 40: Fonction getAll

Source :Auteurs

Cette fonction récupère **toutes les lignes de la table** plan_urbanisme. Elle est souvent utilisée pour afficher l'ensemble des plans enregistrés dans l'interface utilisateur.

- **Fonction getByColumnValue**

```
//récupérer des données en triant les colonnes(certaines valeurs de colonnes en particulier)
exports.getByColumnValue = async (column, value) =>{
  const allowed_column = ['plan_region', 'plan_district', 'plan_commune', 'plan_pude', 'plan_type'];
  if (!allowed_column.includes(column)){
    throw new Error('Colonne non autorisée');
  }
  const query = format(`SELECT * FROM plan_urbanisme WHERE %I = $1`, column);
  const result = await bd.query(query, [value]);
  return result.rows;
}
```

Figure 41: Fonction getByColumnValue

Source :Auteurs

Cette fonction permet de **filtrer les résultats** selon une colonne spécifique (ex : tous les plans d'une région). Pour des raisons de sécurité, seules certaines colonnes sont autorisées. Cela évite des injections SQL ou des requêtes non contrôlées.

- **Fonction addOne**

```
//ajout d'une donnée
exports.addOne = async ({plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type}) =>{
  const result = await bd.query(`INSERT INTO plan_urbanisme
    (plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type)
    VALUES ($1, $2, $3, $4, $5, $6, $7) RETURNING *`,
    [plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type]);
  return result.rows[0];
}
```

Figure 42: Fonction addOne

Source :Auteurs

Cette fonction permet **d'ajouter un nouveau plan** dans la base. Elle prend en paramètre les différentes informations (région, district, commune, etc.) et insère une nouvelle ligne dans la table plan_urbanisme.

- **Fonction deleteById**

```
//suppression d'une donnée
exports.deleteById = async (id) =>{
  const result = await bd.query(`DELETE FROM plan_urbanisme WHERE id_plan = $1 RETURNING *`, [id]);
  return result.rows;
}
```

Figure 43: Fonction deleteById

Source : Auteurs

Cette fonction supprime un plan d'urbanisme à partir de son **identifiant (id_plan)**. Elle retourne les données supprimées, ce qui peut être utile pour confirmer la suppression dans l'interface.

- **Fonction updateById**

```
//modification / mise-à-jour d'une donnée
exports.updateById = async (id_plan, plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type) =>{
  const query = `UPDATE plan_urbanisme
    SET plan_region = $1,
    plan_district = $2,
    plan_commune = $3,
    plan_pude = $4,
    plan_carte = $5,
    plan_rapport = $6,
    plan_type = $7,
    WHERE id_plan = $8`;
  const value = [plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type, id_plan];
  try{
    const result = await bd.query(query, value);
    return result.rows;
  }catch(err){
    throw new Error('Erreur de mise à jour : ' + err.message);
  }
}
```

Figure 44: Fonction updateById

Source :Auteurs

Cette fonction sert à **modifier un enregistrement existant**. Elle prend en entrée l'identifiant du plan et les nouvelles valeurs pour chaque champ. Si une erreur se produit, elle est capturée et renvoyée avec un message personnalisé.

Ce modèle montre comment les différentes opérations CRUD(Create, Read, Update, Delete) sont regroupées dans un seul fichier, ce qui facilite la maintenance du code. Ce modèle est ensuite utilisé dans les **controllers** pour traiter les requêtes venant des routes.

V.2.2.1.2. Controller

Les **controllers** contiennent la **logique métier**. Ce sont eux qui reçoivent les données envoyées par le client (frontend), les traitent, puis appellent les modèles pour interagir avec la base. Un contrôleur peut par exemple vérifier que les champs obligatoires sont bien remplis. En résumé, les contrôleurs agissent comme des "cerveaux" qui organisent ce qu'il faut faire lors d'une requête.

Comme exemple, on prendra le fichier controller de la table plan_urbanisme. Il contient le même nombre de fonctions que le fichier model, c'est-à-dire cinq (5). Évidemment, le fichier model correspondant est importé dans le controller.

- **Fonction findAll**

```
exports.findAll = async (req, res) =>{
  try{
    const findAllPlans = await plan.getAll();
    res.(local var) error: any
  } catch(error){
    res.status(500).json({error: error.message});
  }
};
```

Figure 45: Fonction findAll

Source :Auteurs

Cette fonction récupère **tous les plans d'urbanisme** depuis la base de données.

Elle appelle la méthode **getAll()** du modèle (*plan.getAll()*) pour obtenir un tableau de tous les enregistrements.

Si tout se passe bien, elle renvoie ce tableau au client en JSON (*res.json(findAllPlans)*).

En cas d'erreur, elle renvoie un statut 500 avec un message **d'erreur** (*res.status(500).json({ error: error.message })*).

- **Fonction findByColumnValue**

```
exports.findByColumnValue = async(req, res) =>{
  const {column, value} = req.params;
  try{
    const findOnePlan = await plan.getByColumnValue(column, value);
    res.json(findOnePlan);
  } catch(error) {
    res.status(400).json({error: error.message});
  }
}
```

Figure 46: Fonction findByColumnValue

Source :Auteurs

Cette fonction permet de **filtrer les plans** en fonction d'une colonne et d'une valeur passées en paramètres d'URL.

Elle récupère *column* et *value* depuis *req.params*.

Elle utilise **plan.getByColumnValue(column, value)** pour interroger la base de façon sécurisée.

Si la colonne n'est pas autorisée ou qu'une autre erreur survient, elle renvoie un statut **400 (Bad Request)** avec l'erreur. Sinon, elle renvoie les résultats en JSON.

- **Fonction remove**

```
exports.remove = async (req, res) => {
  try {
    const {id} = req.params;
    await plan.deleteById(id);
    res.json({ message: `Produit avec ID ${id} supprimé.` });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}
```

Figure 47: Fonction remove

Source :Auteurs

Cette fonction supprime un plan identifié par son **ID** :

- L'ID est lu dans req.params.id.
- Elle appelle **plan.deleteById(id)**, qui exécute la commande SQL de suppression.
- En cas de succès, elle renvoie un message de confirmation (`{ message: ... }`) en JSON.
- En cas d'erreur serveur, elle renvoie un statut 500 avec le message d'erreur.

- **Fonction addPlan**

```
exports.addPlan = async (req, res) => {
  try {
    const {plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type} = req.body;
    if (!plan_type || !plan_region || !plan_carte || !plan_rapport) {
      return res.status(400).json({ error: 'Champs requis manquants' });
    }
    const addOnePlan = await plan.addOne({plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type});
    res.json(addOnePlan);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
}
```

Figure 48: Fonction addPlan

Source :Auteurs

Cette fonction crée un **nouveau plan d'urbanisme** :

- Elle lit les champs (*plan_region, plan_district, etc.*) dans **req.body**.
- Elle vérifie que les **champs obligatoires** (plan_type, plan_region, plan_carte, plan_rapport) sont présents ; sinon, elle renvoie un statut 400.
- Si la validation passe, elle appelle plan.addOne(...) pour insérer la nouvelle ligne et renvoie l'objet créé en JSON.
- En cas d'erreur inattendue, elle renvoie un statut 500 et le message d'erreur.

- **Fonction modifyPlan**

```
exports.modifyPlan = async (req, res) => {
  try {
    const id = req.params.id;
    const {plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type} = req.body;
    await plan.updateById(id, plan_region, plan_district, plan_commune, plan_pude, plan_carte, plan_rapport, plan_type)
    res.status(200).json({ message: 'Mise à jour réussie' });
  } catch {
    console.error('Erreur updatePlan:', error);
    res.status(500).json({ error: 'Erreur lors de la mise à jour' });
  }
}
```

Figure 49: Fonction modifyPlan

Source :Auteurs

Cette fonction met à jour un **plan existant** :

- L'identifiant id est récupéré depuis req.params.id.
- Les nouveaux champs sont lus dans req.body.
- Elle appelle plan.updateById(...) pour appliquer les modifications en base.
- Si tout se passe bien, elle renvoie un statut 200 avec un message de succès.
- En cas d'erreur durant la mise à jour, elle capture l'erreur (log en console) et renvoie un statut 500 avec un message générique.

V.2.2.1.3. Route

Les **routes** définissent les différents points d'entrée de l'API. C'est elles qui précisent quelles fonctions sont appelées selon l'URL et la méthode HTTP (GET, POST, PUT, DELETE). Par exemple, lorsqu'on fait une requête GET /api/users, la route associe cette URL à une fonction dans le contrôleur qui va récupérer tous les utilisateurs. Les routes servent donc de "carte" pour orienter les requêtes vers le bon endroit.

Toujours en exemple la table *plane_urbanisme*.

```
const express = require('express');
const router = express.Router();
const planController = require('../controllers/plan_urb.controller');

router.get('/', planController.findAll);
router.get('/filter/:column/:value', planController.findByColumnValue);
router.delete('/delete/:id', planController.remove);
router.post('/add', planController.addPlan);
router.put('/:id', planController.modifyPlan);

module.exports = router;
```

Figure 50: Routes de la table plan_urbanisme

Source :Auteurs

Ce fichier définit les **routes** liées à la table plan_urbanisme. Les routes servent à **recevoir les requêtes HTTP** venant du frontend, et à les **diriger vers les fonctions du contrôleur** approprié (plan_urb.controller.js).

- On importe Express et on crée un router qui permet de gérer des sous-routes.
- On importe le contrôleur planController qui contient toute la logique métier pour les plans.
- La route **get('/')** permet de **récupérer tous les plans d'urbanisme**. Quand le client envoie une requête GET à la racine (/), la fonction *findAll* du contrôleur est appelée. Elle retourne tous les enregistrements de la table plan_urbanisme.
- La route **get('/filter/ :column/ :value)** permet de **filtrer les plans** selon une colonne spécifique (ex : région, district, type).
Les paramètres :column et :value sont récupérés dans l'URL, puis transmis à la fonction du contrôleur pour effectuer une recherche conditionnelle.
- La route **delete('/delete/ :id')** permet de **supprimer un plan** selon son identifiant (id_plan).
Elle appelle la fonction *remove ()* du contrôleur, qui exécute une requête DELETE dans la base.
- La route **post('/add')** sert à **ajouter un nouveau plan d'urbanisme**. Elle attend que les données du plan soient envoyées dans le body de la requête (au format JSON).
La fonction *addPlan()* du contrôleur prend ces données et les insère dans la base.
- La route **put (' :id')** permet de **modifier un plan existant**, en précisant son identifiant dans l'URL.
Les nouvelles valeurs sont fournies dans le corps (req.body), puis traitées par la fonction *modifyPlan()* du contrôleur.

V.2.2.1.4. bd_config

Le fichier bd_config contient les informations sur la base de données :

- Le nom d'utilisateur
- Le nom où la base de données est maintenue
- Le nom de la base de données
- Le mot de passe de l'utilisateur

- Le port sur lequel elle est maintenu

```
module.exports = {
  user: 'postgres',
  host: 'localhost',
  database: 'memoireDB',
  password: 'loicandr',
  port: 5432,
}
```

Figure 51: Configuration de la base de données

Source :Auteurs

V.2.2.2. Front-End côté client

Le frontend de la plateforme a été entièrement reconstruit à l'aide du Framework Vue.js, afin d'améliorer l'expérience utilisateur, la maintenabilité du code, et l'interactivité des pages. Le choix de Vue.js s'est imposé grâce à sa légèreté, sa courbe d'apprentissage douce, et sa compatibilité avec les bonnes pratiques du développement moderne (composants, routing, etc.).

V.2.2.2.1. Structure du projet avec Vue.js :

Le frontend est structuré selon une architecture en composants. Chaque fonctionnalité est encapsulée dans un fichier. Vue. On montrera un aperçu des codes du header et de l'accueil :

- **header.vue**

```
<template>
  <div class="entete">
    <div class="logos"> ...
  </div>
  <div class="barnav"> ...
  </div>
</template>
```

Figure 52: Extrait de code VueJs – Partie HTML

Source :Auteurs

Le header est divisé en deux parties : Espace des logos et Espace des liens
L'espace des logos contient :

- Le logo du Ministère de la Décentralisation et de l'Aménagement du Territoire

- Le logo de la République de Madagascar
- Le logo de l'Observatoire du Territoire

L'espace des liens contiendra la barre de navigation du site.

- **Accueil.vue :**

```

1 <template>
2   <div class="Accueil">
3     <div id="first" class="part">...
101   </div>
102
103   <div id="second" class="part">...
138   </div>
139
140   <div id="third" class="part">...
187   </div>
188
189   <div id="fourth" class="part">
190     <div class="titre">...
193   </div>
194   <div class="row">...
211   </div>
212   <div class="expand">
213     <router-link to="#" class="btn btn-success">Voir Plus</router-link>
215   </div>
216   </div>
217
218   </div>
220 </template>
221
222 <style lang="scss">
223   .Accueil{
224     color: black;
225     margin-top: 40px;
226   }

```

Figure 53 : Extrait de code VueJs de l'accueil

Source : Auteurs

L'accueil est divisé en quatre parties :

- Le bloc 'First' affiche les récentes publications postées dans le site web.
- Le bloc 'Second' liste les services offerts par l'OT.
- Le bloc 'Third' cite les procédures.
- Le bloc 'Fourth' énumère les partenaires de l'OT

V.2.2.2.2. Etablissement des routes :

Il n'y a pas que la structure, mais le routage des pages a aussi été mise en place.

```
import { createRouter, createWebHistory } from 'vue-router';

import Accueil from './pages/Accueil.vue';
import Observatoire from './pages/Observatoire.vue';
import These from './pages/these&memoire/these.vue';
import Publications from './pages/publication.vue';
import Procedures from './pages/procédures/amenagement.vue';
import Topo from './pages/procédures/topo.vue';
import Domaines from './pages/procédures/domaine.vue';
import pudé from './pages/OTP/PUDé.vue';
import pudi from './pages/OTP/PUDI.vue';
import sac from './pages/OTP/SAC.vue';
import srat from './pages/OTP/SRAT.vue';
import snat from './pages/OTP/SNAT.vue';
import saic from './pages/OTP/SAIC.vue';
```

Figure 54: Import des pages dans routes.js

Source :Auteurs

Dans un fichier JavaScript routes.js, nous avons stocké les routes de chaque page vue.

- On importe d'abord chaque page comme on importe un composant (component)
- On définit page par page les routes menant à ces derniers. Par exemple pour aller à la page **Thèse et Mémoire** la route est '*http://localhost :5173/Thèse*'

```
const routes = [
  {path: '/', component: Accueil},
  {path: '/Thèse', component: These},
  {path: '/Observatoire', component: Observatoire},
  {path: '/Publications', component: Publications},
  {path: '/Procédures/Aménagement', component: Procedures},
  {path: '/Procédures/Topo', component: Topo},
  {path: '/Procédures/Domaine', component: Domaines},
  {path: '/Plan-Urbanisme/PUDé', component: pudé},
  {path: '/Plan-Urbanisme/PUDI', component: pudi},
  {path: '/Schemas-Amenagement/SAC', component: sac},
  {path: '/Schemas-Amenagement/SRAT', component: srat},
  {path: '/Schemas-Amenagement/SNAT', component: snat},
  {path: '/Schemas-Amenagement/SAIC', component: saic},
```

Figure 55: Définition des routes des page

Source : Auteurs

V.2.2.2.3. Page outils de planification-PUDé :

La page outils de planification est la page dédiée à présenter les outils de planifications territoriales utiliser à Madagascar. Ces outils sont présentés sous forme de tableau, relié à

une table *plan_urbanisme*. Pour pouvoir comprendre le mécanisme de cette page, prenons comme exemple l'outil de planification PUDé.

Voici le bloc Template du PUDé :

```

1 <template>
2   <div class="pude">
3     <div class="ban">
4       <h5>PUDé (Plan d'Urbanisme Détails)</h5>
5     </div>
6     <div class="container-fluid">
7       <div class="row">
8         <div class="col-md-3">
9           <ResponsiveNav v-model="selected" :categories="menuData" />
10        </div>
11        <div class="col-md-9">
12          <h3>Liste des PUDé</h3>
13          <table class="tableau">
14            <thead>
15              <tr>
16                <th>Région</th>
17                <th>District</th>
18                <th>Commune</th>
19                <th>PUDé</th>
20                <th>Cartographie Thématique & Rapport</th>
21              </tr>
22            </thead>
23            <tbody>
24              <tr v-for="(plan_urbanisme , i) in plans " :key="i">
25                <td>{{ plans.plan_region }}</td>
26                <td>{{ plans.plan_district }}</td>
27                <td>{{ plans.plan_commune }}</td>
28                <td>{{ plans.plan_pude }}</td>
29                <td>{{ plans.plan_raport }}</td>
30              </tr>
31            </tbody>
32          </table>
33        </div>
34      </div>
35    </template>

```

Figure 56 : Template de la page PUDé(extrait)

Source : Auteurs

Le PUDé est divisé en deux parties :

- La classe ban : qui comporte la bannière de l'outil de planification, ici c'est PUDé
- La classe container : qui est le conteneur principal qui comporte le tableau de chaque outil ainsi les types d'outils sous forme de menu.

Le bloc style qui comporte les styles du tableau comme la disposition des éléments, les couleurs

```

66 <style lang="scss">
67   .pude{
68     margin-top: 5vh;
69     margin-bottom: 10vh;
70 >   | .ban{...
88   | }
89   | .col-md-9{
90 >   | | .search{...
120  | | }
121  | }
122 }
123 </style>

```

Figure 57 : CSS de la page PUDé (extrait)

Source : Auteurs

Le bloc script :

```

36 <script>
37 import axios from 'axios';
38 import { onMounted } from 'vue';
39 export default {
40   name: 'pude',
41   data(){
42     return{
43       plan_urbanisme:[],
44       selected :null,
45       menuData:[]
46     }
47   },
48   methods:{
49     async fetchData(){
50       try{
51         const response = await fetch("http://localhost:3000/api/plans/plan_type/pude");
52         const data=await response.json();
53         this.plans=data;
54       } catch(error){
55         console.error("Erreur de changement:",err);
56       }
57     },
58   },
59 },
60 Mounted(){
61   | this.fetchData();
62 }
63 }

```

Figure 58 : Script de la page PUDé (extrait)

Source : Auteurs

Ce fichier sert à afficher des données liées aux plans d'urbanisme de type "pude" " récupérées depuis une API backend.

- Le composant utilise la Composition API de Vue 3 avec le hook onMounted importé.
- Dans la fonction data (), trois propriétés réactives sont définies :
 - Plan_urbanisme : un tableau qui stockera les données des plans d'urbanisme récupérées depuis l'API
 - Selected : une variable pour suivre l'élément sélectionné (valeur initiale null)
 - menuData : un tableau pour stocker d'éventuelles données de menu

- La méthode `fetchData ()` est une fonction asynchrone qui effectue une requête HTTP GET vers la route "http://localhost:3000/api/plans/plan_type/pude" pour récupérer les données des plans qui ont comme type 'PUDE'.
 - Elle utilise l'API Fetch pour récupérer les données
 - Les données JSON sont passées et stockées dans la propriété `plane` du composant
- Le hook `Mounted ()` appelle la fonction `fetchData ()` lorsque le composant est monté dans le DOM, garantissant que les données sont chargées au moment où le composant est rendu.

V.2.2.3. Frontend côté administrateur

Dans le but de la mise-à-jour facile de la base de données, nous avons mis en place une interface admin.

V.2.2.3.1. Page LogIn

```
<template>
  <div class="form-container">
    
    <h2>Log In</h2>
    <form @submit.prevent="login">
      <label>Username :</label>
      <input v-model="username" required />

      <label>Password :</label>
      <input v-model="password" type="password" required />

      <button type="submit">LogIn</button>
    </form>

    <p v-if="message" :class="{error: isError}">{{ message }}</p>
  </div>
</template>
```

Figure 59: Partie HTML du formulaire de login

Source : Auteurs

Comme on le voit sur le code, le formulaire comporte deux champs : Username et Password et un bouton de connexion LogIn. Chacun de ces champs doivent être rempli avant l'envoi du formulaire.

En appuyant sur le bouton, après remplissage des champs, le code exécutera la fonction 'login' dans la partie script de la page vuejs.

```

export default {
  data() {
    return {
      username: "",
      password: "",
      message: ""
    };
  },
  methods: {
    async login() {
      this.message = "";
      try {
        const response = await fetch('http://localhost:3000/api/administrateurs/login', {
          method: 'POST',
          headers: {
            'Content-Type': 'application/json',
          },
          body: JSON.stringify({
            username: this.username,
            password: this.password
          })
        });
        const data = await res.json();
        if (res.ok) {
          localStorage.setItem("token", data.token);
          const redirect = this.$route.query.redirect || "/admin/plans/table";
          this.$router.push(redirect);
        } else {
          this.message = data.error || `Identifications Incorrects`;
        }
      } catch (error) {
        this.message = `Erreur réseau : ${error.message}`;
      }
    },
  },
}

```

Figure 60: Partie JavaScript du formulaire de login

Source : Auteurs

Les informations saisies dans le formulaire sont stockées dans les variables ‘username’ et ‘password’ de **data ()**.

Dans **method ()**, on a défini la fonction « login » qui est utilisé dans le formulaire.

Une fois le formulaire envoyé, le code envoie un requête http au serveur backend, correspondant à la route de connexion, ainsi que le username et password pour la vérification dans la base de données. Il transforme ensuite la réponse du backend en objet JavaScript.

Si la réponse est positive, donc l’username et le password sont corrects, le ‘token’ généré est stocké dans le navigateur pour garder l’utilisateur connecté même s’il recharge la page. Par ailleurs, le code récupère la page où l’admin veut aller, sinon le renvoie à la page des plans d’urbanisation par défaut.

Par contre, si la connexion échoue, le code mettra l’erreur renvoyé au backend dans ‘this.message’.

V.2.2.3.2. Tableau d'affichage

Pour chaque table, un tableau d'affichage a été construite pour la recherche, la suppression ou la modification des données.

On prendra le tableau et les formulaires pour la table plan urbanisme pour montrer la structure des codes.

```
<div class="container">
  <h2>Données dans la table plan_urbanisme</h2>

  <div class="filters">
    <div class="search">
      <label class="search_elm">Champ à filtrer : </label>
      <select v-model="selectedField" class="search_elm">
        <option v-for="key in tableHeaders" :key="key" :value="key">{{ key }}</option>
      </select>

      <input
        class="search_elm"
        type="text"
        v-model="searchQuery"
        placeholder="Rechercher une valeur..."
      />
    </div>
    <div class="add">
      <router-link to="/admin/add"></router-link>
    </div>
  </div>
</div>
```

Figure 61: HTML du tableau d'affichage (Partie 1)

Source : Auteurs

```
<table v-if="filteredItems.length > 0" border="1">
  <thead>
    <tr>
      <th v-for="header in tableHeaders" :key="header">{{ header }}</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody>
    <tr v-for="(item, i) in filteredItems" :key="i">
      <td v-for="header in tableHeaders" :key="header">{{ item[header] }}</td>
      <td>
        <button class="modify" @click="editRow(item.id_plan)">Modifier</button>
        <button class="delete" @click="deleteRow(item.id_plan)">Supprimer</button>
      </td>
    </tr>
  </tbody>
</table>
```

Figure 62: HTML du tableau d'affichage (Partie 2)

Source : Auteurs

On a d'abord le champ de recherche et la sélection de champs à rechercher. Les champs se trouvant dans cette sélection sont prises dynamiquement selon les données affichées. Si, dans la partie script, on modifie le query pour que les données affichées soient de la table 'statistiques', alors la sélection contiendra automatiquement les champs de cette table.

La génération des colonnes du tableau ne sont pas statiques mais dépend aussi des données à affichées.

A chaque ligne se trouvent deux boutons : 'Modifier' et 'Supprimer'.

Passons à la partie script.

```
data() {
  return {
    items: [],
    searchQuery: "",
    selectedField: "",
  };
},
computed: {
  filteredItems() {
    if (!this.selectedField || !this.searchQuery) return this.items;
    return this.items.filter(item =>
      String(item[this.selectedField])
        .toLowerCase()
        .includes(this.searchQuery.toLowerCase())
    );
  },
  tableHeaders() {
    return this.items.length > 0 ? Object.keys(this.items[0]) : [];
  },
}
```

Figure 63: Code de la fonction de recherche

Source : Auteurs

Ici, items contient toutes les données de la table, searchQuery la donnée recherchée et selectedFields le champ où on doit rechercher.

Le but est de retourner le filtrage de items selon les paramètres searchQuery et selectedField. Si aucun champ n'a été sélectionné ou bien s'il n'y a aucune valeur de recherche, le code retournera toutes les données.

Sinon le code filtre le tableau et retourne les données correspondant à la recherche.

```

methods: {
  async fetchData() {
    try {
      const res = await fetch("/api/plans");
      const data = await res.json();
      this.items = data;
    } catch (err) {
      console.error("Erreur de chargement :", err);
    }
  },
  async editRow(id_plan){
    this.$router.push({name: "modifTable", params:{id_plan : id_plan}});
  },
  async deleteRow(id_plan) {
    if (!confirm("Voulez-vous vraiment supprimer cette ligne ?")) return;
    try {
      const res = Response
      const res = Response
    });
    if (!res.ok) throw new Error("Échec de la suppression.");
    this.items = this.items.filter((item) => item.id_plan !== id_plan);
  } catch (err) {
    console.error("Erreur lors de la suppression :", err);
  }
},
},
mounted() {
  this.fetchData();
}

```

Figure 64: Code d’affichage, de modification et de suppression

Source : Auteurs

On a créé trois fonctions principales pour cette page : la fonction `fetchData`, la fonction `editRow` et la fonction `deleteRow`.

- **fetchData** : elle exécute la route du backend correspondant à la récupération des données et les stocke dans la constante ‘items’.
- **editRow** : c’est la fonction qui s’exécute lorsque le bouton ‘Modifier’ est cliqué. Il redirige l’utilisateur vers un formulaire de modification tout en mettant en paramètre l’id de la donnée à modifier.
- **deleteRow** : c’est la fonction qui s’exécute lorsque le bouton ‘Supprimer’ est cliqué. Il affiche d’abord une alerte, et si on confirme, il exécute la route du backend correspondant à la suppression de données par rapport à l’id en paramètre.

V.2.2.3.3. Formulaire d'ajout

```
<div class="form-container">
  <h2>Ajouter un plan d'urbanisme</h2>
  <form @submit.prevent="submitForm">
    <label>Région :</label> (property) region: string
    <input v-model="region" required />

    <label>District :</label>
    <input v-model="district" required />

    <label>Commune :</label>
    <input v-model="commune" required />

    <label>PUDE :</label>
    <input v-model="pude" required />

    <label>Type :</label>
    <select v-model="type" required>
      <option disabled value="">-- Sélectionnez un type --</option>
      <option v-for="t in typesList" :key="t" :value="t">{{ t }}</option>
    </select>

    <label>Carte :</label>
    <input v-model="carte" required />

    <label>Rapport :</label>
    <input v-model="rapport" required />

    <button type="submit">Ajouter</button>
  </form>

  <p v-if="message">{{ message }}</p>
</div>
```

Figure 65: Partie HTML du formulaire d'ajout

Source : Auteurs

Chaque input ici correspond à un champ de la table plan_urbanisme. Et la fonction 'submitForm' est celle qui sera enclenchée après avoir cliqué sur le bouton 'Ajout'.

```

export default {
  data() {
    return {
      region: '',
      district: '',
      commune: '',
      pude: '',
      carte: '',
      rapport: '',
      type: '',
      message: '',
      typesList: ['PUDé', 'PUDI'],
    };
  },
}

```

Figure 66: Javascript du formulaire d'ajout (Partie 1)

Source : Auteurs

```

methods: {
  async submitForm() {
    try {
      const response = await fetch('http://localhost:3000/api/plans/add', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          plan_region: this.region,
          plan_district: this.district,
          plan_commune: this.commune,
          plan_pude: this.pude,
          plan_carte: this.carte,
          plan_rapport: this.rapport,
          plan_type: this.type,
        }),
      });

      const data = await response.json();
      if (response.ok) {
        this.message = 'Plan urbanisme ajouté avec succès!';
        this.region = this.district = this.commune = this.pude = this.type = this.carte = this.rapport = '';
      } else {
        this.message = `Erreur : ${data.error}`;
      }
    } catch (error) {
      this.message = `Erreur réseau : ${error.message}`;
    }
  },
},
}

```

Figure 67: Javascript du formulaire d'ajout (Partie 2)

Source : Auteurs

Les variables dans Data () représentent les champs du formulaire. Par exemple *région* est le champ avec '*région*' comme **v-model**.

La fonction d'ajout est comme suit :

- Elle exécute la route du backend correspondant à l'ajout d'un nouveau plan d'urbanisme.
- Comme paramètre, elle y insère les valeurs rentrées dans le formulaire.

- Une fois l'ajout fait, un message de réussite s'affiche, sinon c'est un message d'erreur.

V.2.2.3.4. Formulaire de modification

```
<div class="form-container">
  <h2>Modifier un plan d'urbanisme</h2>
  <form @submit.prevent="modifPlan">
    <label>Région :</label>
    <input v-model="region" placeholder="modifData.plan_region" required />

    <label>District :</label>
    <input v-model="district" placeholder="modifData.plan_district" required />

    <label>Commune :</label>
    <input v-model="commune" placeholder="modifData.plan_commune" required />

    <label>PUDE :</label>
    <input v-model="pude" placeholder="modifData.plan_pude" required />

    <label>Type :</label>
    <select v-model="type" required>
      <option disabled value="">-- Sélectionnez un type --</option>
      <option v-for="t in typesList" :key="t" :value="t">{{ t }}</option>
    </select>

    <label>Carte :</label>
    <input v-model="carte" placeholder="modifData.plan_carte" required />

    <label>Rapport :</label>
    <input v-model="rapport" placeholder="modifData.plan_rapport" required />

    <button type="submit">Enregistrer</button>
  </form>
```

Figure 68: Partie HTML du formulaire de modification

Source : Auteurs

Chaque champ du formulaire est le même que celui d'ajout, mais les valeurs initiales des données à modifier sont mise en placeholder.

En confirmant la modification, la fonction modifPlan s'enclenche.

```

export default {
  props: [":id_plan"],
  data() {
    return {
      region: '',
      district: '',
      commune: '',
      pude: '',
      carte: '',
      rapport: '',
      type: '',
      message: '',
      typesList: ['PUDé', 'PUDI'],
    };
  },
  async fetchData() {
    const res = await fetch(`http://localhost:3000/api/plans/id_plan/${this.id_plan}`);
    const modifData = await res.json();
  },

```

Figure 69: Javascript du formulaire de modification (Partie 1)

Source : Auteurs

```

methods: {
  async modifPlan() {
    try {
      const response = await fetch(`http://localhost:3000/api/plans/${this.id_plan}`, {
        method: 'PUT',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          plan_region: this.region,
          plan_district: this.district,
          plan_commune: this.commune,
          plan_pude: this.pude,
          plan_carte: this.carte,
          plan_rapport: this.rapport,
          plan_type: this.type,
        }),
      });

      const data = await response.json();
      if (response.ok) {
        this.message = 'Plan urbanisme ajouté avec succès!';
        this.region = this.district = this.commune = this.pude = this.type = this.carte = this.rapport = '';
      } else {
        this.message = `Erreur : ${data.error}`;
      }
    } catch (error) {
      this.message = `Erreur réseau : ${error.message}`;
    }
  },

```

Figure 70: Javascript du formulaire de modification (Partie 2)

Source : Auteurs

Les données de base sont d'abord prises dans la base de données avec route du backend correspondant à la recherche des données par valeur de colonne (findByColumnValue) et sont mises en placeholder dans le formulaire.

Tout comme pour le formulaire d'ajout, les nouvelles données sont placées dans les variables de Data (), sont mises en paramètres et la route du backend pour la modification est exécutée.

Chapitre VI : Interface et Fonctionnement du Logiciel

VI.1. Extrait du Côté Client

VI.1.1. Guide :

Dès que l'utilisateur ouvre le lien du site, il accèdera à cette page, qui est en guise de guide de navigation. L'utilisateur aura alors un aperçu de ce que le site représente, même si c'est la première fois qu'il le consulte.

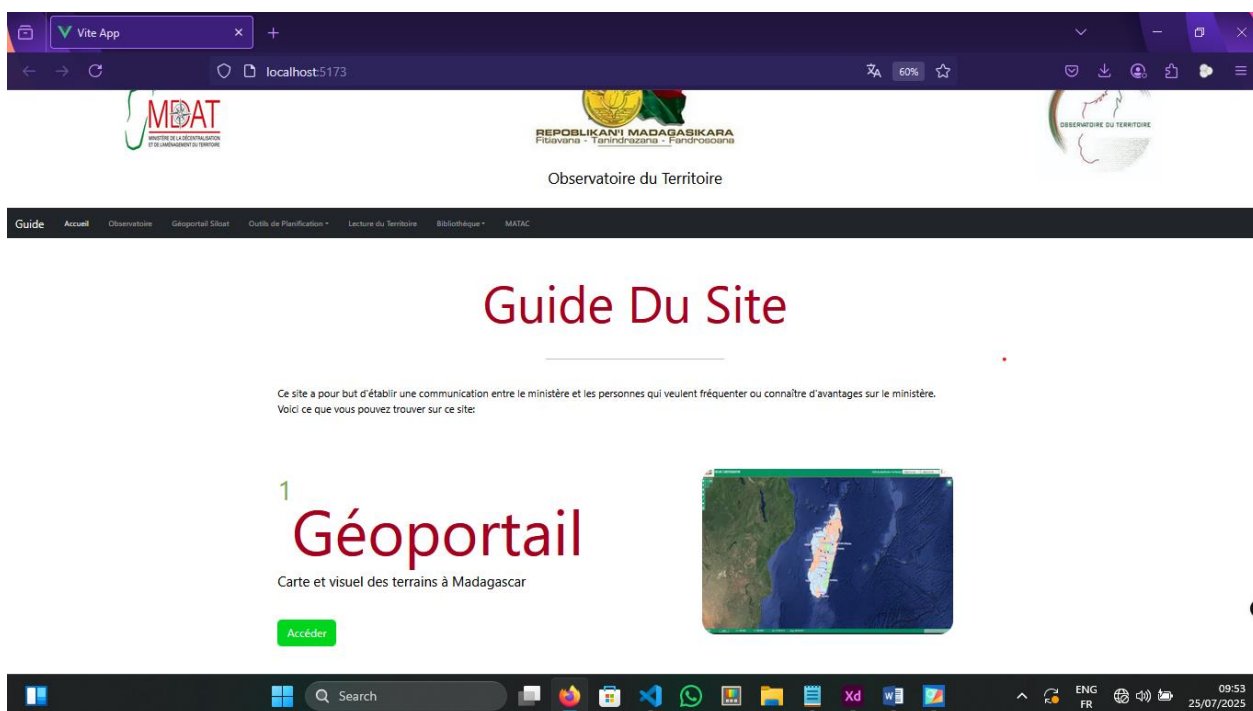


Figure 71: Interface de la page Guide

Source :Auteurs

En consultant ce guide, l'utilisateur peut :

- Savoir les descriptions du contenu de chaque page
- Voir les pages, les outils et les fichiers qui se trouve dans le site
- Accéder directement aux pages, aux outils et aux documents via le bouton '*accéder*'

VI.1.2. Outils de planifications :

VI.1.2.1. Présentation

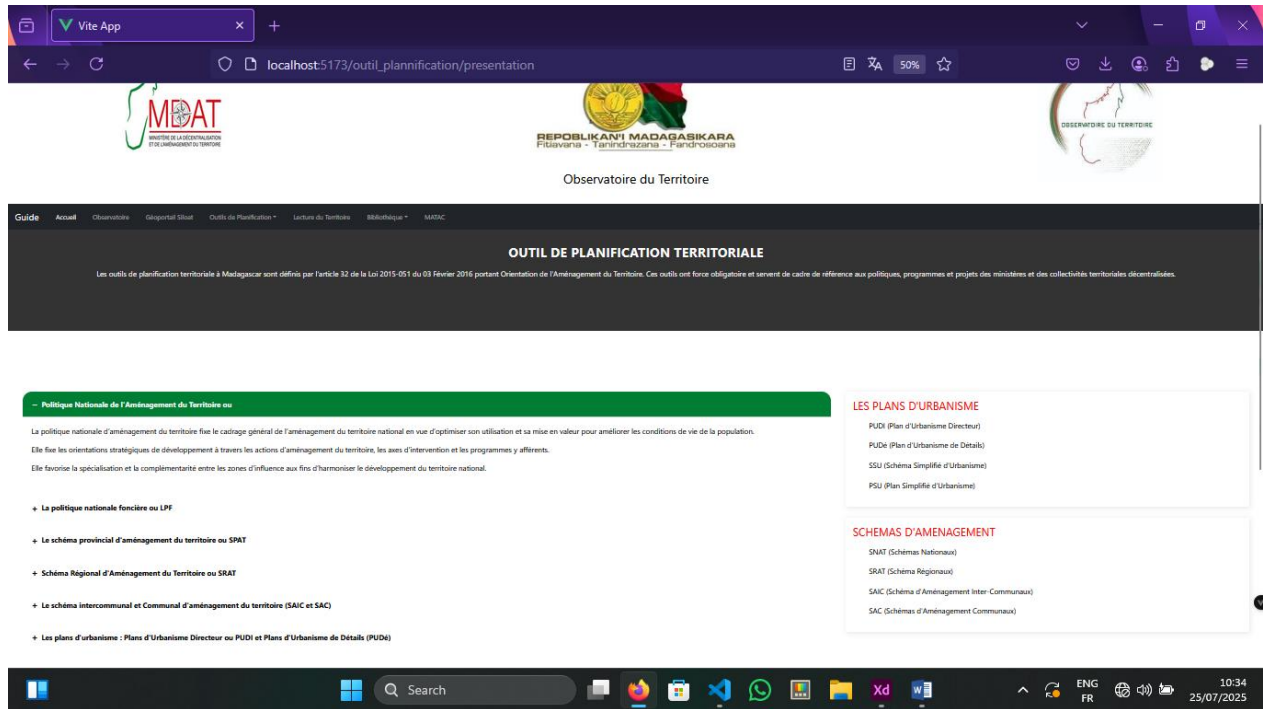


Figure 72: Interface de la page Outils de Planification

Source :Auteurs

Cette page renseigne l'utilisateur à propos des outils de planifications territoriales. Chaque outil est représenté sous forme d'accordéon. Et lorsqu'on y clique, les informations à propos de l'outil choisi s'affiche.

VI.2.2.2. Statistiques

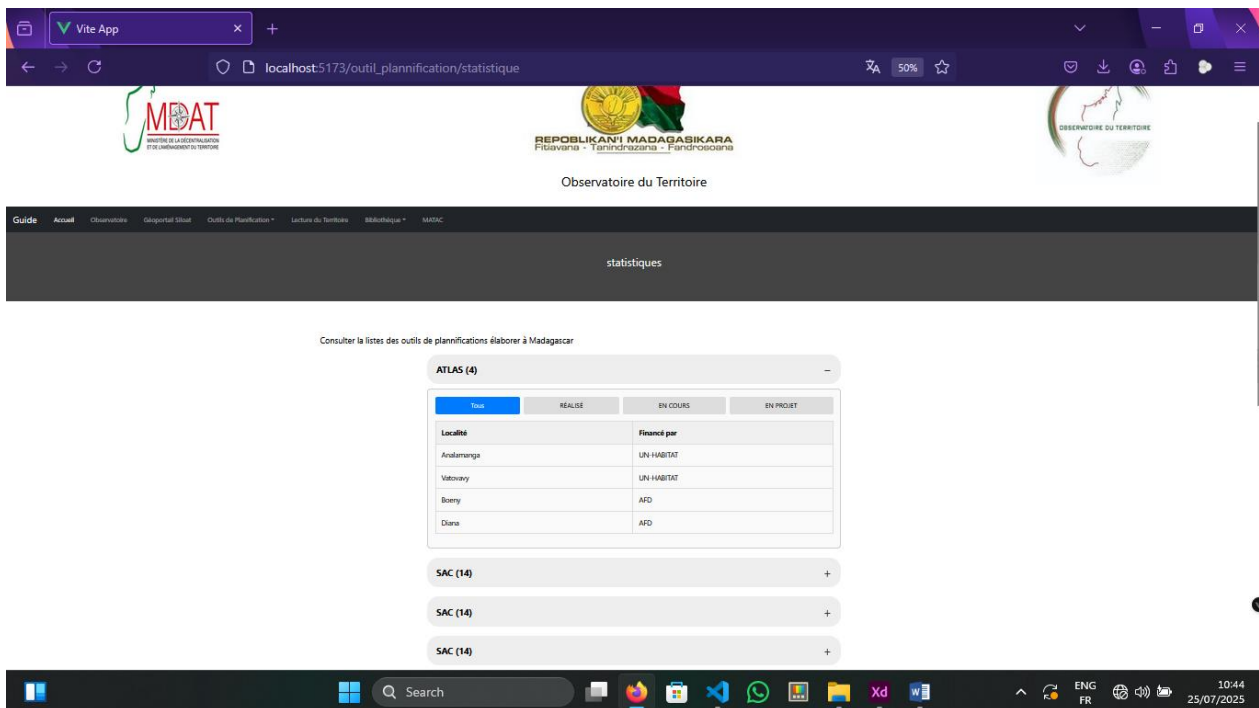


Figure 73: Interface de la page Statistiques

Source :Auteurs

Cette page comprend les outils de planification élaboré à Madagascar, présentés sous forme de boutons déroulants. Lorsqu'on clique sur un bouton un tableau s'affiche.

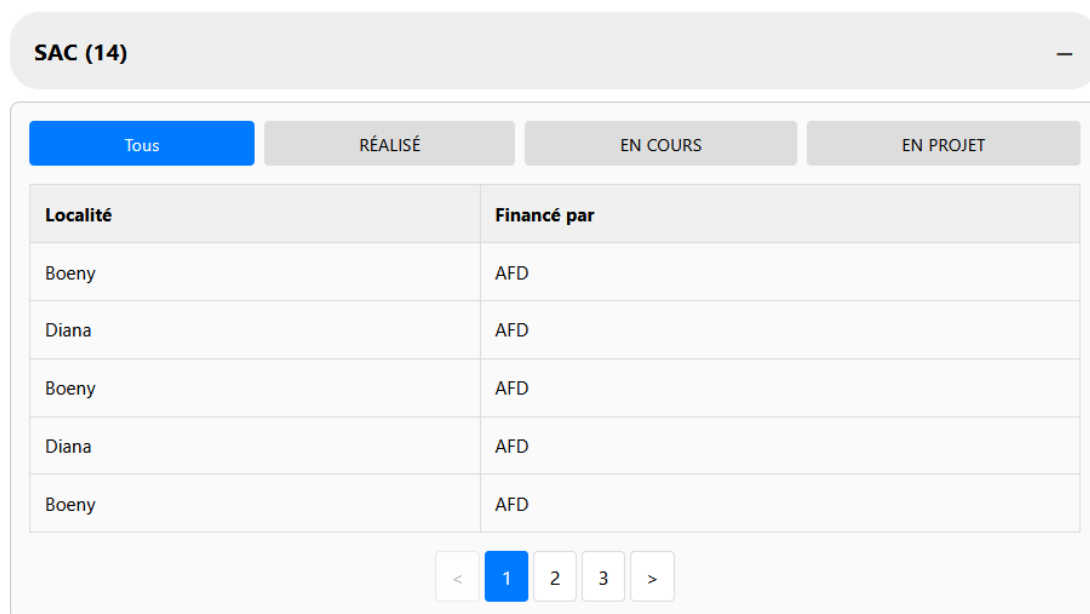


Figure 74: Contenu de chaque bouton déroulant

Source :Auteurs

Pour l'outil SAC par exemple, les informations peuvent être filtrées. Par défaut, le tableau affiche tous les projets dans SAC. En appuyant sur le bouton 'REALISE', il n'affichera que les projets dont le statut est 'réalisé'. Ainsi de suite pour les boutons 'EN COURS' et 'EN PROJET'.

Le tableau possède également un système de pagination dans le cas où les projets d'un outil sont nombreux.

VI.2. Côté Administrateur

VI.2.1. Authentification :

Si nous voulons accéder à la partie administration de la plateforme, l'administrateur devra d'abord s'identifier avec son username et son password.

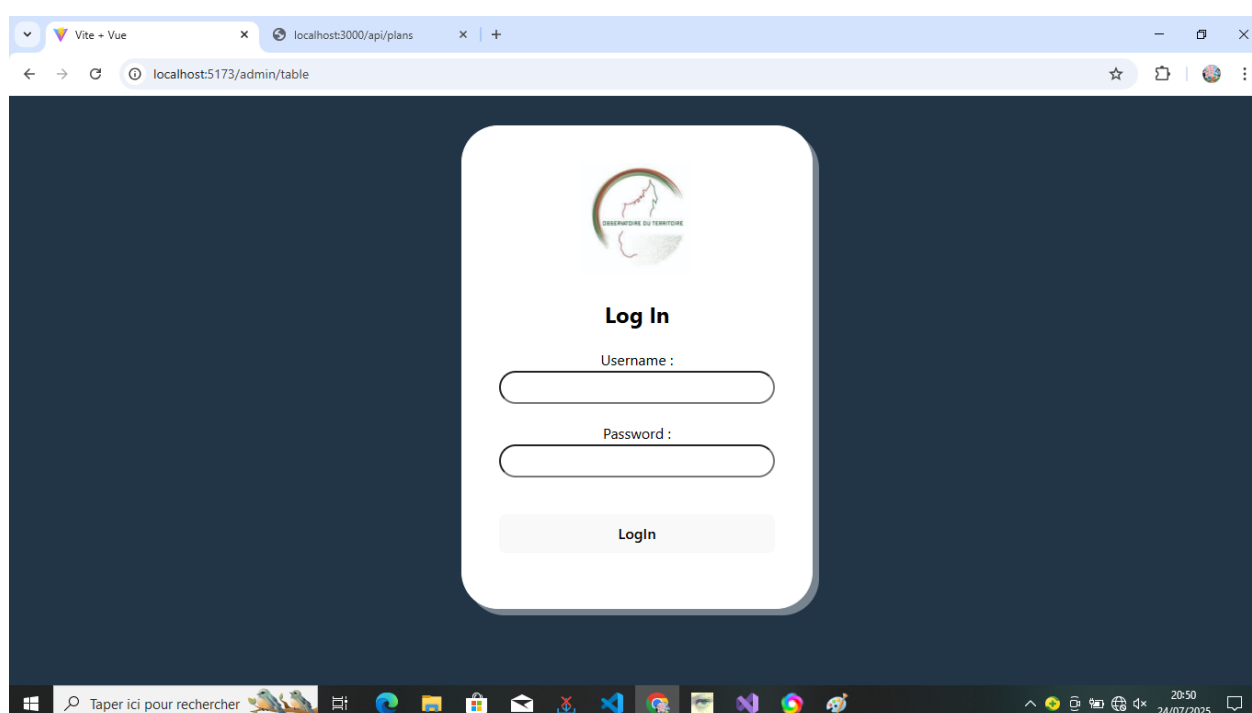


Figure 75: Connexion à l'admin

Source :Auteurs

Dans le cas où l'identifiant ou le mot de passe ne soit pas enregistré dans la base de données ou comporte une erreur cette identification échoue et un message d'erreur apparaît.

Sinon, on accèdera aux pages de modification des données

VI2.2. Manipulation des données

Données dans la table plan_urbanisme

Champ à filtrer : Rechercher une valeur...

id_plan	plan_region	plan_district	plan_commune	plan_pude	plan_carte	plan_rapport	plan_type	Action
0bb189fd-b4c1-4d66-ac59-cde8795d8afb	region100	dis2	com2	pude1	carte2	rapport2	pude	<button>Modifier</button> <button>Supprimer</button>
1302a51d-65f5-4453-8178-d8e5559f8db0	reg200	dis3	com3		carte3	rapport3	pudi	<button>Modifier</button> <button>Supprimer</button>
5682a9ea-cc7c-454a-8a82-ecf2b3e102f5	reg5	dis500	com500		carte500	rapport500	PUDI	<button>Modifier</button> <button>Supprimer</button>
5231db46-								<button>Modifier</button>

Administrateurs
Plan Urbanisme
Procédures
Publications
Schémas Aménagement
Statistiques
Textes de Lois
Thèses et Mémoires

Déconnexion 20:32 24/07/2025

Figure 76: Données dans la table plan_urbanisme

Source :Auteurs

A droite, nous avons la barre de navigation pour changer de tables :

- Administrateurs pour les données de la table admins
- Plan urbanisme pour la table plan_urbanisme.
- Procédures pour la table procedure.
- Publications pour la table publication
- Schémas Aménagement pour la table schemas_reg
- Textes de Lois pour la table texte_lois
- Thèses et Mémoires pour la table these_mem

Après avoir sélectionnée la table à consulter, les champs et données de cette dernière s'affichera sous forme de tableau avec un barre de recherche, un bouton d'ajout, de modification et de suppression.

Nous allons illustrer le fonctionnement de l'interface en utilisant Plan urbanisme comme exemple.

VI.2.3. Recherche

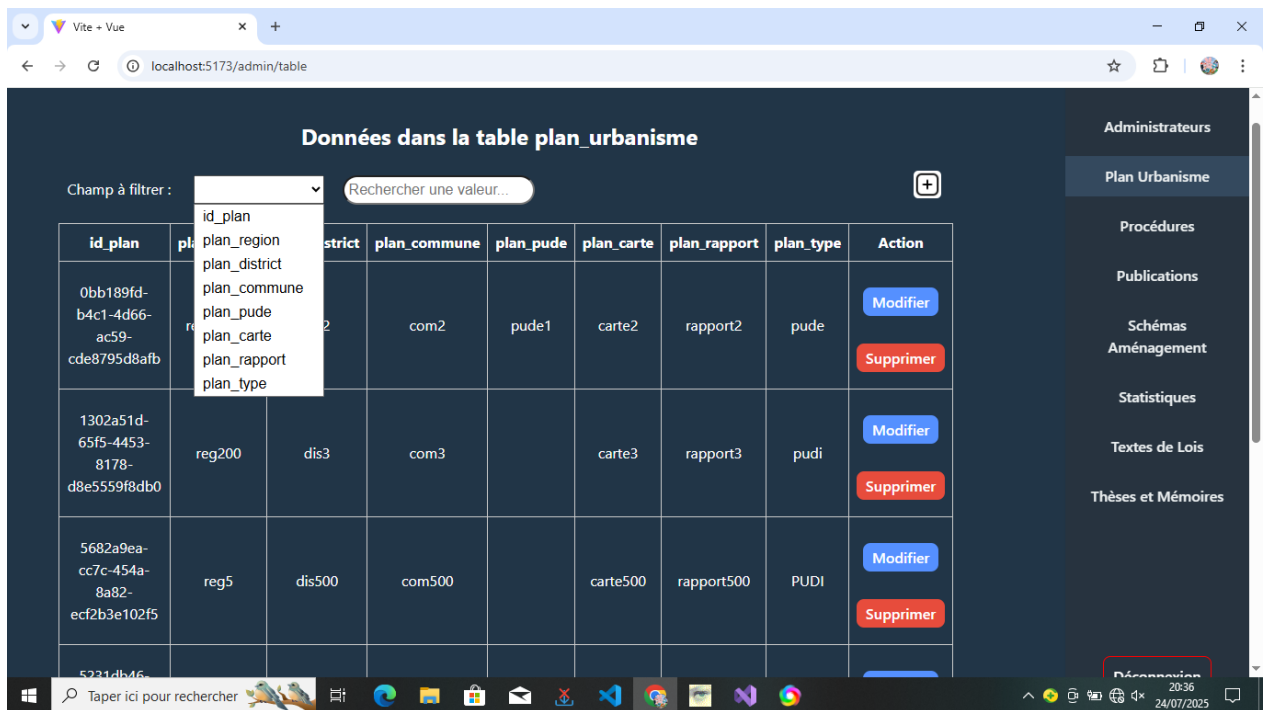


Figure 77: Champs de recherche

Source :Auteurs

En haut du tableau, nous avons une barre de recherche :

- On sélectionne d'abord la colonne où on veut faire la recherche.
- Sur la zone de texte, on entrera la valeur que l'on veut chercher.
- Le tableau se triera automatiquement selon la valeur recherchée.

VI.2.4. Modification

The screenshot displays a web browser window with the address bar showing 'localhost:5173/admin/table'. The main content area features a form titled 'Modifier un plan d'urbanisme'. The form includes the following fields and values:

- Région : region100
- District : dis2
- Commune : com2
- PUDE : pude1
- Type : PUDs (dropdown menu)
- Carte : carte2
- Rapport : rapport2

Below the fields is a button labeled 'Enregistrer'. To the right of the form is a dark sidebar with a list of navigation items: Administrateurs, Plan Urbanisme, Procédures, Publications, Schémas Aménagement, Statistiques, Textes de Lois, and Thèses et Mémoires. The 'Plan Urbanisme' item is highlighted. The Windows taskbar at the bottom shows the search bar and various application icons.

Figure 78: Formulaire de modification

Source :Auteurs

En cliquant sur le bouton *Modifier*, un formulaire se chargera avec, comme valeur de départ dans les champs, les données de la ligne.

On effectue les modifications souhaitées, puis ensuite, après avoir appuyé sur le bouton *Enregistrer*, le formulaire enregistrera les nouvelles données entrées.

VI.2.5. Suppression

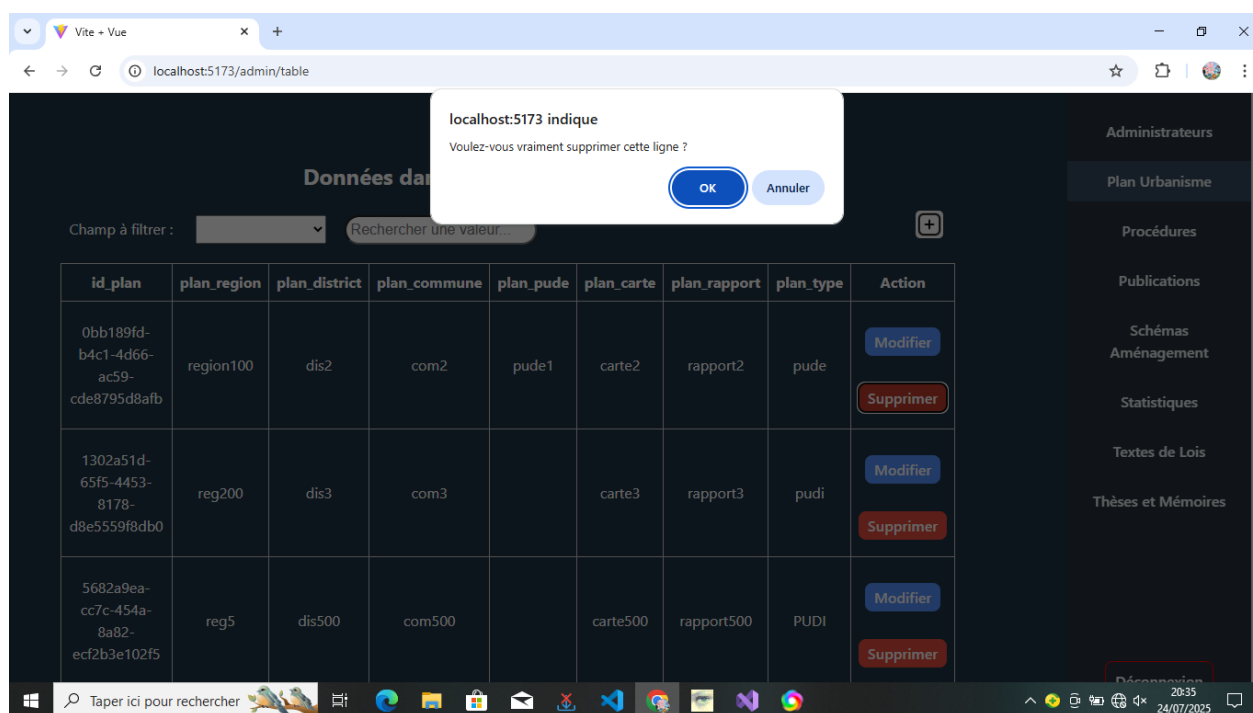


Figure 79: Confirmation de suppression

Source :Auteurs

En appuyant sur le bouton Supprimer, un message d'alerte s'affiche qui confirmera la suppression de la ligne.

Après confirmation, la ligne sera supprimée du tableau et de la base de données définitivement.

VI.2.6. Ajout

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/admin/add'. The page has a dark blue background. In the center, there is a white rounded rectangle containing the form titled 'Ajouter un plan d'urbanisme'. The form includes the following fields: 'Région :', 'District :', 'Commune :', 'PUDE :', 'Type :', 'Carte :', and 'Rapport :'. The 'Type :' field is a dropdown menu with the text '-- Sélectionnez un type --'. At the bottom of the form is a button labeled 'Ajouter'. On the right side of the page, there is a dark blue sidebar with a list of menu items: 'Administrateurs', 'Plan Urbanisme', 'Procédures', 'Publications', 'Schémas', 'Aménagement', 'Statistiques', 'Textes de Lois', and 'Thèses et Mémoires'. The Windows taskbar is visible at the bottom of the screen.

Figure 80: Ajout de données

Source :Auteurs

Le bouton '+' en haut à droite du tableau générera un formulaire. Ce dernier est utilisé pour l'ajout de données dans la table.

Après le remplissage des champs et en appuyant sur le bouton *Ajouter*, le formulaire ajoutera les nouvelles données à la table et au tableau.

Au cas où un champ obligatoire n'est pas rempli, une notification s'affichera au niveau de ce champ.

PARTIE III : RÉSULTATS ET DISCUSSIONS

Chapitre VII : Approches Qualitatives du Projet

La refonte du site web a nécessité une révision complète de l'architecture technique, tant du backend que du frontend. L'objectif principal était d'offrir une application web moderne, fluide, sécurisée et évolutive. Cette approche full-stack a permis de repenser non seulement les traitements côté serveur avec Express.js et PostgreSQL, mais aussi toute l'expérience utilisateur grâce à un frontend développé avec VueJS.

VII.1. Choix des technologies utilisées :

Pour le frontend, nous avons choisi d'utiliser VueJS, un framework JavaScript. Ce choix a été encouragé par son système d'utilisation de composants (components) réutilisables, qui non seulement est très pratique sur la structure des codes, mais aussi fait éviter de créer plusieurs structures de codes à même structures mais à contenu différents.

Le backend a été entièrement reconstruit avec le framework ExpressJS, qui fonctionne avec NodeJS. On a choisi ce framework pour sa facilité, sa capacité de créer des API REST de manière simple et claire mais aussi pour sa bonne compatibilité avec VueJS. De plus, il fonctionne bien avec JavaScript, ce qui nous a permis de rester dans l'utilisation du même langage durant tout ce projet.

Quant à la base de données, le projet initial utilisait déjà PostgreSQL comme système de gestion. Dans le cadre de la refonte, il a été décidé de conserver cette technologie. Ce choix a plusieurs raisons : tout d'abord, la base de données existante était fonctionnelle et contenait déjà des données structurées. Il n'était donc pas nécessaire de la migrer vers un autre système, ce qui aurait entraîné un travail supplémentaire sans réel bénéfice. Ensuite, PostgreSQL est un SGBD performant et conforme aux standards SQL, largement utilisé dans le monde professionnel. Il convient parfaitement aux besoins de l'application, notamment pour assurer l'intégrité des données. Conserver PostgreSQL a permis de réduire le temps de développement, d'éviter les problèmes de compatibilité, et de se concentrer sur la réécriture du backend. Aussi pas besoin de reconstruire la base de données lors du déploiement du projet.

VII.2. Résultats et Améliorations Observés :

VII.2.1. Côté Client :

La nouvelle interface, développée avec **Vue.js**, a été pensée pour offrir une navigation plus fluide et intuitive. Contrairement à l'ancienne version qui présentait un design figé et

parfois désorganisé, la nouvelle mouture repose sur une architecture orientée composants, ce qui a permis de gagner en clarté et en modularité.

- **Clarté de l'interface** : les informations sont désormais mieux hiérarchisées, avec des titres lisibles, des icônes explicites et des couleurs cohérentes avec l'identité institutionnelle.
- **Navigation simplifiée** : grâce au système de routing de Vue.js, les transitions entre pages sont instantanées, sans rechargement complet, améliorant considérablement la fluidité.
- **Accessibilité améliorée** : une attention particulière a été portée au responsive design. Les tests sur différents terminaux (ordinateur, smartphone, tablette) ont confirmé que l'interface s'adapte bien à la majorité des résolutions standards.

Exemples de fonctionnalités bien perçues :

- Le nouveau tableau interactif avec tri dynamique
- Le formulaire de contact avec validation instantanée des champs
- La section "Qui sommes-nous ?" repensée avec une disposition plus aérée

VII.2.2. Côté Serveur :

La refonte du backend a permis d'apporter plusieurs améliorations concrètes en comparaison avec l'ancienne version du site. Ces changements ont concerné à la fois la structure du code, la sécurité, la performance, et la facilité de maintenance.

- **Structure claire et modulaire** : la structure MVC a rendu le code plus lisible, maintenable et évolutif, ce qui facilite l'ajout de nouvelles fonctionnalités.
- **Séparation des tâches** : chaque composant du backend a une responsabilité précise. Par exemple, les fonctions de vérification de l'authentification ont été isolées dans des middlewares, ce qui permet de les réutiliser facilement sur plusieurs routes. Cette séparation permet aussi de mieux gérer les erreurs.
- **Création d'API RESTful** : l'API construite permet d'interagir proprement avec les données (création, lecture, mise à jour, suppression). Cela facilite la communication avec le frontend développé en Vue.js, et permettrait même à l'avenir d'utiliser d'autres clients (par exemple une application mobile) sans avoir à modifier le backend.

VII.3. Limites Remarquées :

Comme tout projet, celui-ci contient aussi des limites :

- On doit utiliser une gestion de stockage autre que la base de données (comme Google Drive) pour stocker les documents (en PDF) mis en exposition sur le site.
- Il n'y a pas encore de tests automatisés, tout est testé manuellement avec Postman et dans le navigateur.
- L'état global du frontend n'est pas géré avec Vuex ou Pinia, ce qui pourrait devenir un problème si l'application devient plus complexe.

Malgré cela, le site est fonctionnel, et l'ensemble du système backend + frontend communique correctement.

Chapitre VIII : Futures Améliorations du Logiciel

Le développement de la nouvelle version de la plateforme représente une base solide, mais plusieurs améliorations restent envisageables pour assurer sa montée en charge et sa compatibilité avec de nouveaux besoins institutionnels.

VIII.1. Améliorations côté client :

- **Utiliser un gestionnaire d'état global** : Pour le moment, le partage de données entre composants (comme l'utilisateur connecté) se fait de manière basique. Pour un projet plus gros, il serait préférable d'utiliser Vuex ou Pinia pour centraliser les données et éviter les répétitions.
- **Intégration d'une intelligence artificielle** : Une intégration d'une intelligence artificielle serait utile. Elle pourrait aider les utilisateurs à naviguer sur le site, poser des questions fréquentes, ou automatiser un support client basique. Elle pourrait aussi proposer à l'utilisateur des contenus, documents ou services en fonction de son historique ou de son comportement. Par exemple, donner suggestions basées sur les données de navigation (les pages les plus consultées, etc).
- **Accessibilité et UX** : Le site pourrait être rendu plus accessible aux personnes ayant un handicap (ex. : navigation au clavier, contrastes renforcés, balises ARIA).

VIII.2. Améliorations backend :

- **Tests automatisés** : Ajouter des tests unitaires et fonctionnels permettrait de vérifier automatiquement que l'API fonctionne bien après chaque modification. Cela augmenterait la fiabilité du code sur le long terme.
- **Mise en cache** : Pour les routes les plus consultées, on pourrait utiliser un système de cache afin de réduire la charge sur la base de données.
- **Côté administrateur en espace employé** : L'interface administrateur ici ne sert qu'à la manipulation de la base de données. Dans le futur, elle pourrait être transformée en espace pour les employés de l'OT. Par exemple, on y inclura un système de présence, une gestion des tâches journalières ou hebdomadaire, un suivi des projets en cours, un système de communications entre employé.

VIII.3. Déploiement :

Actuellement, le projet fonctionne en local. Pour aller plus loin, il faudrait le déployer sur un serveur (par exemple : Render, Vercel, Netlify ou un VPS). Cela permettrait :

- D'utiliser HTTPS pour sécuriser les échanges,
- De tester le comportement réel en ligne,
- D'avoir un système en place pour montrer le site à des utilisateurs ou recruteurs.

On pourrait aussi mettre en place une **chaîne CI/CD** pour automatiser le déploiement après chaque modification du code.

VIII.4. Nouvelles fonctionnalités possibles :

Voici quelques idées de nouvelles fonctionnalités qui pourraient être ajoutées :

- **Un tableau de bord avec statistiques** : il montrera les activités de chaque utilisateur dans le côté administrateur. Par exemple un historique des modifications des données, un historique de connexion, etc.
- Un système de **fichiers joints** ou d'images à uploader : établir une gestion de stockage de fichiers propre au site et éviter d'utiliser une externe.
- Une **messagerie interne** simple : un moyen de communication entre les utilisateurs sur leurs activités en cours ou terminées.

Le projet, dans son état actuel, est déjà un bon départ. Il est fonctionnel, sécurisé et bien structurée. Mais comme tout projet en informatique, les possibilités d'améliorations sont nombreuses. Ce que l'on a réalisé est une simple version viable, qu'on pourrait faire évoluer dans le futur.

CONCLUSION

Ce projet avait pour objectif principale de refondre et d'optimiser une plateforme web afin d'améliorer son ergonomie, ces performances techniques et sa compatibilité avec les standards actuels. Cette démarche s'inscrivait dans un contexte où les utilisateurs attendent des interfaces intuitives et rapides, adaptées à leurs besoins.

Grâce à une approche méthodique, la refonte a permis de moderniser l'interface utilisateur. La compatibilité mobile, auparavant limitée, a été optimisée, offrant une meilleure expérience sur tous les appareils. Bien que les objectifs principaux aient été atteints, certaines fonctionnalités, comme l'intégration d'une API de personnalisation n'ont pas pu être entièrement développées dans le cadre du projet. Cela représente une piste de travail pour l'avenir.

En conclusion, ce projet de refonte et d'optimisation a démontré l'importance d'adapter les plateformes web aux exigences modernes. Il constitue une base solide pour des évolutions futures, garantissant une meilleure expérience utilisateur et une visibilité accrue. Dans un monde en constante évolution numérique, ces démarches doivent être perçues comme un processus continu et non comme une finalité.

BIBLIOGRAPHIE

Cours dispenses à l'ISPM :

- [1] RABOANARY Julien Amédée : Gestion de qualité, 2022-2023, L3
- [2] RAKOTOARISOA Hasina Eric; HTML, 2020-2021, L1
- [3] RAKOTOARISOA Hasina Eric; CSS, 2020-2021, L1
- [4] RAKOTOARISOA Hasina Eric; JavaScript, 2021-2022, L2
- [5] RASANDIMANANA Tanjona : Base de données, 2020-2021, L1
- [6] RAZAFIMANJATO Irina Maeva: UI / UX, 2022-2023, L3

Livres et Ouvrages :

- [7] Décret n°2024-049 du 20 janvier 2024 organigramme MDAT

Webographie :

- [8] Bootstrap: <https://getbootstrap.com/>
- [9] NodeJS: <https://nodejs.org/en>
- [10] VueJs: <https://vuejs.org/>
- [11] ExpressJs: <https://expressjs.com/>
- [12] PostgreSQL : <https://www.postgresql.org/>
- [13] Inspiration de Design : <https://fr.pinterest.com/>
- [14] <https://www.w3schools.com/>
- [15] <https://www.scribd.com/document/867490168/4-Organigramme-ISPM>
- [16] https://www.flaticon.com/fr/icone-gratuite/html-5_5968267?term=html&page=1&position=3&origin=search&related_id=5968267
- [17] https://www.flaticon.com/fr/icone-gratuite/css-3_5968242?term=css&page=1&position=4&origin=search&related_id=5968242
- [18] <https://techicons.dev/icons/bootstrap>
- [19] https://www.flaticon.com/fr/icone-gratuite/js_5968292?term=js&page=1&position=1&origin=search&related_id=5968292
- [20] https://www.flaticon.com/fr/icone-gratuite/nodejs_919825?term=nodejs&page=1&position=1&origin=search&related_id=919825
- [21] <https://techicons.dev/icons/vuejs>
- [22] <https://imgbin.com/png/sPTfjwH2/node-js-javascript-react-logo-express-js-png>
- [23] https://www.flaticon.com/fr/icone-gratuite/serveur-sql_4492311?term=sql&page=1&position=7&origin=search&related_id=4492311
- [24] <https://1000logos.net/postgresql-logo/>
- [25] <https://techicons.dev/icons/vscode>
- [26] https://www.flaticon.com/fr/icone-gratuite/xd_5968559?term=adobe+xd&page=1&position=86&origin=search&related_id=5968559
- [27] <https://techicons.dev/icons/vuejs>

LETTRE D'APPROBATION



Le Président du Jury,

Professeur RABOANARY Julien Amédée

LU ET APPROUVE PAR



L'encadreur Professionnel

Monsieur RAJAONARISON Eli Olivier



L'encadreur Pédagogique

Monsieur RAJAONERA Christiano Andriamialy

TABLE DES MATIÈRES

REMERCIEMENTS

AVANT-PROPOS

LISTE DES ANNEXES

LISTE DES ABRÉVIATIONS

LISTE DES FIGURES

LISTE DES TABLEAUX

SOMMAIRE

INTRODUCTION.....	1
-------------------	---

PARTIE I : CADRE D'ETUDE ET DE TRAVAIL

Chapitre I : A propos de l'ISPM.....	2
I.1. Historique :.....	2
I.2. Objectif de l'ISPM :.....	5
I.3. Identification et activités :.....	6
I.4. Coursus à l'ISPM :.....	7
I.5. Performances et prix obtenus par l'ISPM.....	10
I.6. Organigramme de l'ISPM :.....	11
Chapitre II : A propos de la Direction au sein du MDAT.....	12
II.1. Présentation De L'Observatoire du Territoire (OT).....	12
II.2. Les Missions.....	123
II.2.1. Les Missions des Observatoires.....	123
II.2.1. Les Missions de l'Observatoire du Territoire.....	124
Chapitre III : Présentation et Choix du Projet.....	15
III.1. Contexte et Problématique.....	15
III.1.1. Design.....	15
III.1.2. Non-adaptabilité :.....	19
III.1.3. Mise à jour manuelle :.....	19
III.2. Objectifs du Projet.....	19
III.3. Justification du Choix du Projet.....	20
III.4. Résultats Attendues :.....	20

PARTIE II : CONCEPTION ET DÉVELOPPEMENT

Chapitre IV : Conception du Projet.....	21
IV.1. Langage de modélisation unifié (UML).....	21
IV.1.1. Diagramme de Cas d'Utilisation.....	21
IV.1.1. Diagramme de Classe.....	22

IV.1.1. Diagramme de Séquence.....	23
IV.2. Conception de l'interface utilisateur (Frontend)	24
IV.2.1. Objectifs de la refonte frontend	24
IV.2.2. Outils et technologies utilisés pour la conception.....	24
IV.2.3. Principes de design retenus.....	25
IV.2.4. Extraits des maquettes	26
IV.2.4.1. Maquette de la page d'accueil	26
IV.2.4.2. Maquette de la page des outils de planification	27
IV.2.4.3. Maquette version mobile (responsive)	28
IV.2.5. Conclusion de la phase de conception	29
IV.3. Conception du Backend.....	29
IV.3.1. Objectifs.....	29
IV.3.2. Choix Technologique	30
IV.3.3. Rôles	30
IV.3.4. Structure adoptée	30
Chapitre V : Développement du Projet.....	311
V.1. Les technologies utilisées.....	311
V.1.1. Langage et Framework.....	311
V.1.1.1. HTML et CSS :	311
V.1.1.2. Bootstrap :	322
V.1.1.3. JavaScript.....	333
V.1.1.4. Node.JS	344
V.1.1.5. Vue.JS	355
V.1.1.6. Express.JS	355
V.1.1.7. SQL	366
V.1.1.8. PostgreSQL	377
V.1.1.9. Syntactically Awesome StyleSheets.....	388
V.1.2. Logiciels et Extensions	388
V.1.2.1. Visual Studio Code.....	388
V.1.2.2. Adobe XD	39
V.1.2.3. pgAdmin 4.....	400
V.1.2.4. Vetur.....	411
V.2. Structure de la Base de Données et Extraits de code	422
V.2.1. Structure de la Base de Données	422
V.2.1.1. Table plan_urbanisme	433

V.2.1.2. Table schema_reg.....	444
V.2.1.3. Table procédures	455
V.2.1.4. Table publications.....	466
V.2.1.5. Table stats.....	477
V.2.1.6. Table these_mem.....	488
V.2.1.7. Table texte_lois	49
V.2.1.8. Table admins	500
V.2.2. Extraits de code.....	511
V.2.2.1. Back-End.....	511
V.2.2.1.1. Model.....	522
V.2.2.1.2. Controller.....	544
V.2.2.1.3. Route.....	577
V.2.2.1.4. bd_config.....	58
V.2.2.2. Structure du projet avec Vue.js :.....	59
Chapitre VI : Interface et Fonctionnement du Logiciel.....	68
VI.1.....	Côté Client
73	
VI.2.1. Guide :.....	73
VI.2.2. Outils de planifications :	74
VI.2.2.1. Présentation.....	74
VI.2.2.2. Statistiques	75
VI.2. Côté Administrateur	76
VI.2.1. Authentification :.....	76
VI.2.2. Manipulation des données.....	77
VI.2.3. Recherche.....	78
VI.2.4. Modification.....	79
VI.2.5. Suppression	80
VI.2.6. Ajout.....	81
PARTIE III : RÉSULTATS ET DISCUSSIONS	
Chapitre VII : Approche Qualitative du Projet.....	82
VII.1. Renforcement des fonctionnalités :.....	81
VII.2. Renforcement des fonctionnalités :.....	69
VII.3. Renforcement des fonctionnalités :.....	71
Chapitre VIII : Futures Améliorations du Logiciel	85
VIII.1. Améliorations côté client :.....	85
VIII.2. Améliorations backend :	85

VIII.3. Déploiement :	86
VIII.. Nouvelles Fonctionnalités Possibles :	85
CONCLUSION	87

BIBLIOGRAPHIE

TABLE DES MATIÈRES

ATTESTATION DE STAGE

CURRICULUM VITAE

LETTRE D'APPROBATION

ABSTRACT

RESUME

ABSTRACT

This thesis presents a project focused on the redesign and optimization of an existing web platform, carried out as part of the requirements for the Bachelor's degree in Computer Science. The main objective was to modernize the user interface, optimize performance, and enhance the overall navigation experience, while ensuring better code maintainability.

The initial version of the platform suffered from several limitations, including an outdated interface design, poor usability, and a rigid architecture that hindered scalability. To address these issues, a complete overhaul of the frontend was undertaken using the Vue.js framework, known for its lightweight nature, component-based structure, and ease of integration into modern web development environments.

The adopted methodology was structured around key stages: requirements analysis, system design, component modeling, functional prototyping, interface development, and testing. The redesigned version now offers a responsive, intuitive, and cross-platform interface, following best practices in modern frontend development.

This project highlights the importance of considering user needs when redesigning a web service. By applying an appropriate technical approach, we were able to improve the overall user experience of the platform. The work carried out can also serve as a foundation for adding new features or for planning a broader deployment, for example within a public administration or organization.

Key words: *redesign, optimization, web platforms, user experience, project management, responsive design, web development.*

RESUME

Ce mémoire porte sur la refonte et l'optimisation de la plateforme web de l'Observatoire du Territoire, mené dans le cadre de l'obtention de la licence en Informatique. L'objectif principal était de moderniser l'interface utilisateur, optimiser les performances et améliorer l'expérience de navigation, tout en assurant une meilleure maintenabilité du code. A travers ce projet, nous avons exploré les étapes de construction d'une plateforme web, de l'analyse des besoins des utilisateurs tout en renforçant nos compétences en gestion de projet et en les appliquant dans le monde professionnel.

La version initiale de la plateforme souffrait de plusieurs limitations, notamment une interface peu ergonomique, un design obsolète et une architecture difficilement extensible. Pour résoudre ces problèmes, une refonte complète du frontend a été entreprise en utilisant le framework Vue.js, reconnu pour sa structure orientée composants et sa facilité d'intégration.

La méthodologie adoptée s'est articulée autour de plusieurs étapes : analyse des besoins, conception de l'architecture de la plateforme, modélisation des composants, élaboration de maquettes fonctionnelles, développement de l'interface et phase de tests. La nouvelle version possède désormais une interface responsive, intuitive, compatible avec les différents supports (PC, mobile, tablette), et structurée selon les bonnes pratiques du développement web moderne.

Ce projet montre l'importance de prendre en compte les besoins des utilisateurs lors de la refonte d'un service web. En utilisant une approche technique adaptée, nous avons pu améliorer l'expérience globale de la plateforme. Le travail réalisé peut également servir de base pour ajouter d'autres fonctionnalités ou envisager un déploiement plus large, par exemple dans une administration ou une structure publique.

Mots clés : *refonte, optimisation, plateforme web, expérience utilisateur, gestion de projet, interface responsive, développement web.*